
Ingeniería de Datos Moderna

Domina el data engineering moderno con Python, Airflow y dbt

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

Ingeniería de Datos Moderna

Domina el data engineering moderno con Python, Airflow y dbt

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de este libro puede ser reproducida, distribuida o transmitida en cualquier forma o por cualquier medio, incluyendo fotocopiado, grabación u otros métodos electrónicos o mecánicos, sin el permiso previo por escrito del autor.

Colección Data & Analytics Mastery

Libro 4 de 5

Contacto:

alexgoyzueta2018@gmail.com

TechStore es una empresa ficticia creada con fines educativos. Cualquier similitud con empresas reales es mera coincidencia.

Versión: 1.0 — Julio 2026

Dedicatoria

A los ingenieros de datos que construyen silenciosamente la infraestructura que hace posible el análisis, el BI y la inteligencia artificial.

Sin pipelines no hay dashboards. Sin calidad no hay confianza. Sin orquestación no hay escala.

Prefacio

Los cuatro libros anteriores de esta colección te llevaron desde los fundamentos del análisis de datos (Libro 1) hasta Python y BigQuery en la nube (Libro 2), y luego al Business Intelligence con Power BI y modelado dimensional (Libro 3). En cada uno, TechStore fue el hilo conductor.

Pero hay un problema: todos esos libros asumen que los datos ya están limpios, disponibles y accesibles. En el mundo real, esa es la parte más difícil.

Ahí entra la **Ingeniería de Datos**.

Este libro te enseña a construir la infraestructura que hace posible todo lo demás: pipelines robustos, data lakes, transformaciones confiables con dbt, orquestación con Apache Airflow, contenedores con Docker, y CI/CD para datos.

TechStore crece en este libro. Ya no es solo una base de datos SQLite. Ahora tiene un data lake con datos en Parquet, un pipeline de transformación con dbt, una orquestación completa con Airflow, y todo corre en contenedores con despliegue automatizado.

Si los libros anteriores te enseñaron a *consumir* datos, este te enseña a *producirlos* a escala.

Empecemos.

Introducción

Para quién es este libro

Este libro es para ti si:

- Eres analista de datos y quieres entender cómo funcionan los pipelines por debajo
- Has usado BigQuery, Power BI o dbt y quieres orquestar todo con Airflow
- Quieres aprender data engineering moderno con herramientas open source
- Tienes datos en archivos, bases de datos o APIs y necesitas construir tuberías confiables

Lo que necesitas

- Python 3.10+ instalado
- Docker Desktop (para MinIO, Airflow, bases de datos)
- Git
- Un editor de código (VS Code recomendado)
- 8 GB+ de RAM (16 GB ideal para Airflow + Docker)

El proyecto TechStore

TechStore evoluciona. En Libro 2 migró a BigQuery. En Libro 3 se modeló como star schema. Ahora TechStore tiene un **data lake** completo en MinIO (S3 local), transformaciones con dbt, orquestación con Airflow, y todo corre en contenedores.

```
TechStore Data Platform
├─ Data Lake (MinIO) ── Datos crudos en Parquet
├─ dbt ──────────── Transformaciones (Bronze → Silver → Gold)
├─ Airflow ───────── Orquestación y scheduling
├─ Docker ───────── Contenedores y servicios
└─ CI/CD ────────── Despliegue automatizado
```

Estructura del libro

4 proyectos, 15 capítulos, 4 checkpoints:

Proyecto 1: Fundamentos — Conceptos, Python avanzado, formatos de datos, data lakes

Proyecto 2: dbt — Transformaciones, modelado, testing, documentación

Proyecto 3: Airflow — Orquestación, DAGs, integración, monitoreo

Proyecto 4: Producción — Docker, CI/CD, proyecto final

Cada capítulo termina con ejercicios. Los checkpoints integran todo lo aprendido.

Convenciones

Código en este formato: comandos, SQL, Python, configuración

Negrita: términos importantes, nombres de archivos, herramientas

Cursiva: énfasis, términos en inglés

¡Al grano!

Capítulo 1: Introducción al Data Engineering

¿Qué es Data Engineering?

Data Engineering es la disciplina que diseña, construye y mantiene la infraestructura y los pipelines que transforman datos crudos en datos listos para análisis.

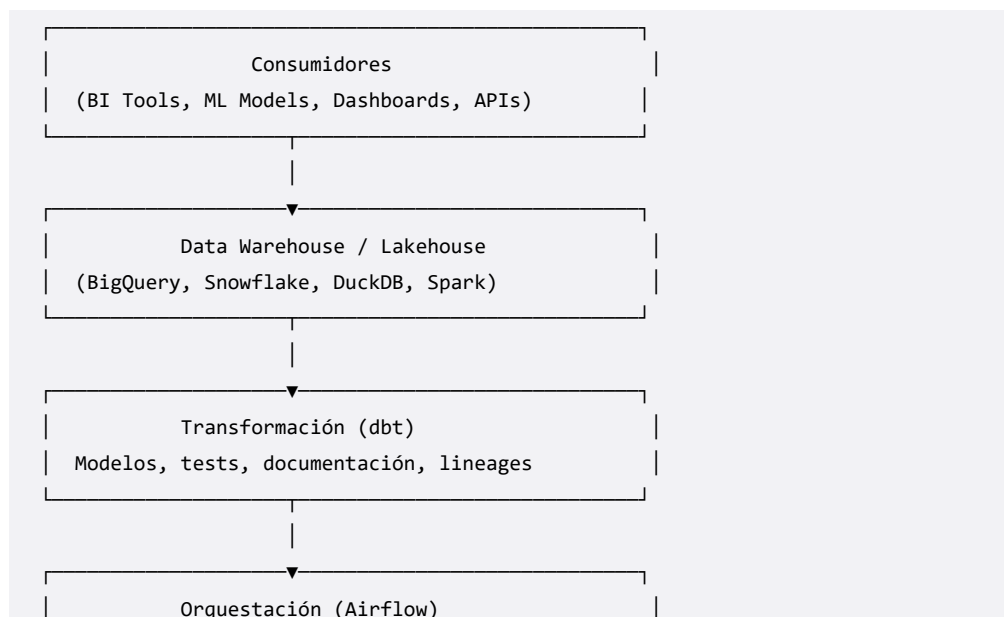
Datos Crudos → Pipeline → Datos Listos → Analistas / BI / ML
(fuentes) (DE) (confiables) (consumidores)

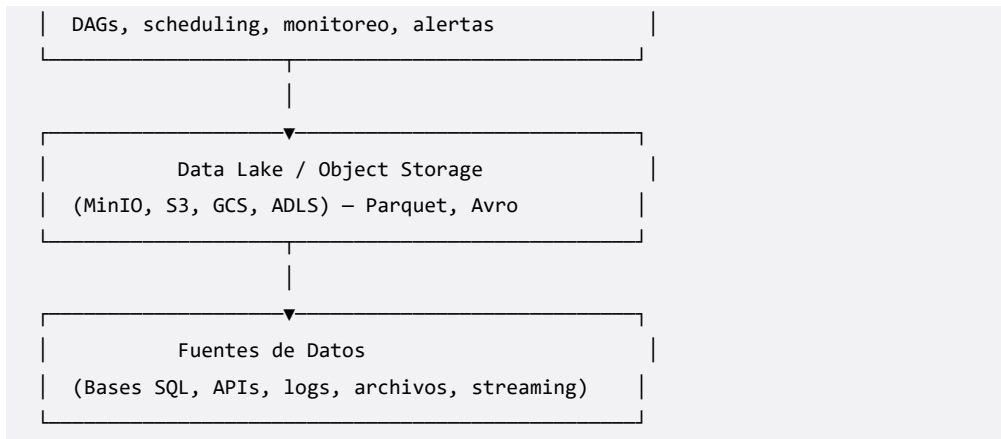
Data Engineering vs Data Science vs Data Analysis

RoI	¿Qué hace?	Herramientas
Data Engineer	Construye pipelines, orquesta, garantiza calidad	Python, SQL, Airflow, dbt, Docker
Data Analyst	Explora, visualiza, genera insights	SQL, Excel, Power BI, Looker
Data Scientist	Modela, predice, experimenta	Python, ML, estadística, Jupyter
Analytics Engineer	Puente entre DE y DA: transforma datos con	SQL, dbt, Python

El stack moderno de Data Engineering

En 2026, el stack moderno se ve así:



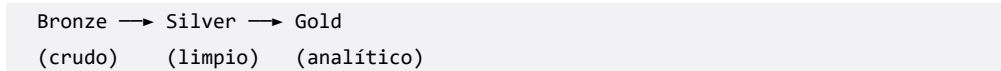


Herramientas de este libro

Herramienta	Propósito
Python	Lenguaje principal para pipelines y scripts
SQL	Consultas, transformaciones, dbt models
MinIO	Data Lake local compatible con S3 API
Parquet / Avro	Formatos columnar/binario eficientes
dbt	Transformaciones, testing, documentación
Airflow	Orquestación, scheduling, monitoreo
Docker	Contenedores para entornos reproducibles
Git + GitHub Actions	CI/CD para pipelines de datos

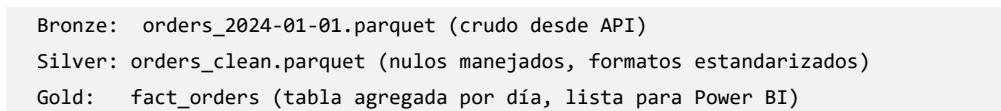
La medallion architecture (Bronze Silver Gold)

El patrón más usado hoy para organizar datos en un data lake:



Capa	Descripción	Formato	Dueño
Bronze	Datos tal cual llegan de la fuente	Parquet (crudo)	Data Engineer
Silver	Datos limpios, validados, deduplicados	Parquet (particionado)	Data Engineer
Gold	Datos agregados, modelados para negocio	tablas SQL / vistas	Analytics Engineer

Ejemplo TechStore



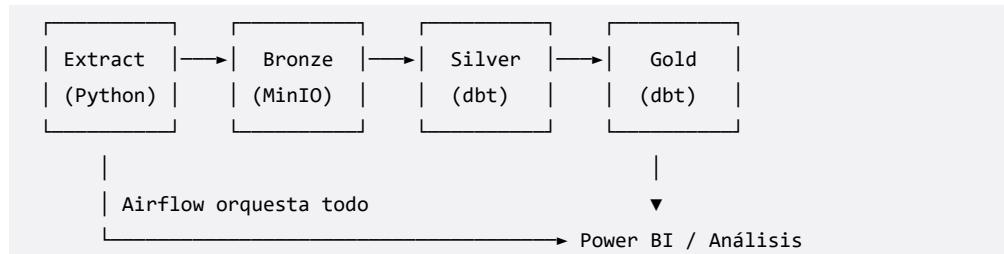
Pipelines: batch vs streaming

Característica	Batch	Streaming
Frecuencia	Diaria, horaria	Tiempo real (segundos)
Volumen	Alto por lote	Continuo, menor por unidad

Complejidad	Baja-media	Alta
Herramientas	Airflow, dbt	Kafka, Flink, Spark Streaming
Uso en TechStore	Reportes diarios	No aplica (por ahora)

En este libro trabajamos con **batch** (el 90% de los casos reales).

El pipeline de TechStore en este libro



Ejercicios

1. ¿Cuál es la diferencia entre Data Engineer y Data Analyst?
2. ¿Qué capa de la medallion architecture contiene datos crudos?
3. ¿Por qué Parquet es mejor que CSV para un data lake?
4. Investiga qué es DuckDB y cómo se compara con SQLite
5. ¿En qué casos usarías streaming en lugar de batch?
6. ¿Qué rol juega dbt en el stack moderno?
7. ¿Qué es un pipeline de datos? Define sus 3 etapas principales
8. ¿Cuál es la ventaja de la medallion architecture frente a un solo contenedor de datos?
9. ¿Cómo encaja Airflow en el stack de TechStore?
10. ¿Qué herramienta de este libro usas ya? ¿Cuál te gustaría aprender más?

Capítulo 2: Python para Data Engineering

Python es el lenguaje del data engineer

No por moda, sino por pragmatismo: Python tiene bibliotecas para todo lo que un data engineer necesita —conexión a bases de datos, procesamiento de archivos, APIs, cloud, orquestación.

En este capítulo repasamos el Python que realmente usas como data engineer.

Estructura de un proyecto

```
techstore_pipeline/  
├── pipelines/           # Código de pipelines  
│   ├── __init__.py  
│   ├── extract.py      # Extracción de fuentes  
│   ├── load.py         # Carga a data lake  
│   └── utils.py        # Utilidades compartidas  
├── config/             # Configuración centralizada  
│   └── config.yaml  
├── tests/  
│   └── test_pipeline.py  
├── requirements.txt  
├── Dockerfile  
└── .env                 # Variables de entorno
```

Configuración centralizada

Nunca hardcodees credenciales o rutas. Usa configs y variables de entorno:

```
# config/config.yaml  
database:  
  host: localhost  
  port: 5432  
  name: techstore  
  user: ${DB_USER}  
  password: ${DB_PASSWORD}  
  
data_lake:  
  endpoint: http://localhost:9000  
  bucket: techstore-bronze
```

```
access_key: ${MINIO_ACCESS_KEY}
secret_key: ${MINIO_SECRET_KEY}
```

```
# config/loader.py
import yaml
import os
from pathlib import Path

def load_config(config_path: str = "config/config.yaml") -> dict:
    with open(config_path, "r") as f:
        config = yaml.safe_load(f)
    return _resolve_env_vars(config)

def _resolve_env_vars(obj):
    if isinstance(obj, dict):
        return {k: _resolve_env_vars(v) for k, v in obj.items()}
    if isinstance(obj, str) and obj.startswith("${") and obj.endswith("}"):
        env_var = obj[2:-1]
        return os.environ.get(env_var, "")
    return obj
```

Logging profesional

`print()` no escala. Usa logging:

```
import logging
import sys

def setup_logger(name: str, level: int = logging.INFO) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(level)

    handler = logging.StreamHandler(sys.stdout)
    handler.setLevel(level)

    formatter = logging.Formatter(
        "%(asctime)s | %(name)s | %(levelname)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S"
    )
    handler.setFormatter(formatter)

    if not logger.handlers:
        logger.addHandler(handler)

    return logger

# Uso
logger = setup_logger("techstore.pipeline")
logger.info("Iniciando extracción de datos...")
logger.warning("No se encontraron datos para la fecha 2024-06-01")
logger.error("Error de conexión a la base de datos: timeout")
```

Manejo de errores robusto

Los pipelines fallan. Diseña para que fallen bien:

```
import time
from functools import wraps

def retry(max_attempts: int = 3, delay: int = 2):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    logger.error(f"Intento {attempt}/{max_attempts} falló: {e}")
                    if attempt == max_attempts:
                        raise
                    time.sleep(delay * attempt)
            return None
        return wrapper
    return decorator

@retry(max_attempts=3, delay=2)
def extract_orders(date: str) -> list[dict]:
    # Simula extracción con posible fallo
    pass
```

Trabajo con archivos y rutas

```
from pathlib import Path

BASE_DIR = Path(__file__).parent.parent
DATA_DIR = BASE_DIR / "data"
BRONZE_DIR = DATA_DIR / "bronze" / "orders"
SILVER_DIR = DATA_DIR / "silver"

# Crear estructura
BRONZE_DIR.mkdir(parents=True, exist_ok=True)

# Listar archivos por fecha
for parquet_file in BRONZE_DIR.glob("orders_*.parquet"):
    print(f"Procesando: {parquet_file.name}")
```

Fechas y particionamiento

Los pipelines casi siempre particionan por fecha:

```
from datetime import datetime, timedelta, date

def get_partition_path(base_path: Path, dt: date) -> Path:
    return base_path / f"year={dt.year}" / f"month={dt.month:02d}" / f"day={dt.day:02d}"
```

```
def date_range(start: date, end: date):
    current = start
    while current <= end:
        yield current
        current += timedelta(days=1)

# Particionar por fecha
start = date(2024, 1, 1)
end = date(2024, 12, 31)
for dt in date_range(start, end):
    path = get_partition_path(BRONZE_DIR, dt)
    print(f"{dt}: {path}")
```

Pipeline extract simple

```
import sqlite3
import pandas as pd
from pathlib import Path

def extract_from_sqlite(db_path: str, table: str, output_path: Path) -> Path:
    logger = setup_logger("extract")
    logger.info(f"Extrayendo {table} de {db_path}")

    conn = sqlite3.connect(db_path)
    df = pd.read_sql_query(f"SELECT * FROM {table}", conn)
    conn.close()

    output_path.parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(output_path, index=False)

    logger.info(f"Extraídas {len(df)} filas -> {output_path}")
    return output_path

# Uso
extract_from_sqlite(
    db_path="data/techstore.db",
    table="orders",
    output_path=BRONZE_DIR / "orders_2024-01-01.parquet"
)
```

Ejercicios

1. Crea un logger con formato `[fecha] [nivel] [módulo] mensaje`
2. Escribe una función `load\_config` que lea un YAML y resuelva variables de entorno
3. Crea un decorador `@retry` que reintente 5 veces con backoff exponencial
4. Escribe una función que liste los archivos Parquet de un directorio particionado
5. ¿Por qué es importante usar `Path` en lugar de strings para rutas?
6. Crea un script que extraiga `customers` de SQLite y lo guarde como Parquet
7. ¿Qué ventaja tiene la configuración centralizada frente a variables sueltas?

8. Implementa un pipeline que extraiga `order_items` y lo guarde particionado por fecha
9. Escribe una función `safe_execute` que maneje excepciones y loguee el error
10. ¿Por qué usar `functools.wraps` en un decorador?

Capítulo 3: Formatos de Datos — Parquet, Avro y CSV

El problema de CSV

CSV es el formato más común y el peor para data engineering:

Problema	CSV	Parquet
Tipo de datos	Todo string	Tipado explícito (int, float, date, etc.)
Compresión	Ninguna (texto plano)	Columnar con compresión (Snappy, Zstd)
Tamaño	10× más grande	Compacto
Lectura	Carga completa	Solo columnas necesarias
Schema	No tiene	Incrustado en el archivo
Particionamiento	Manual	Nativo (Hive-style)

```
# Comparación de tamaño
import pandas as pd

df = pd.read_sql_query("SELECT * FROM orders", conn)

df.to_csv("orders.csv", index=False)      # ~2.5 MB
df.to_parquet("orders.parquet", index=False) # ~350 KB

# ¡7× más pequeño!
print(f"CSV: {os.path.getsize('orders.csv') / 1024:.0f} KB")
print(f"Parquet: {os.path.getsize('orders.parquet') / 1024:.0f} KB")
```

Parquet

Parquet es un formato columnar de código abierto, diseñado para análisis.

Columnar vs row-oriented

```
Row-oriented (CSV, Avro):
order_1 | cust_1 | 2024-01-01 | 150.00 | completed
order_2 | cust_2 | 2024-01-01 | 89.99  | completed
order_3 | cust_1 | 2024-01-02 | 250.00 | pending

Column-oriented (Parquet):
order_id:  order_1,  order_2,  order_3
```

customer:	cust_1,	cust_2,	cust_1
date:	2024-01,	2024-01,	2024-01-02
total:	150.00,	89.99,	250.00
status:	completed,	completed,	pending

Ventaja: si solo necesitas `total`, solo lees esa columna. En CSV lees todo.

Schema de Parquet

```
import pyarrow as pa

schema = pa.schema([
    ("order_id", pa.string()),
    ("customer_id", pa.int64()),
    ("order_date", pa.date32()),
    ("total", pa.float64()),
    ("status", pa.string()),
])

# Escribir con schema explícito
table = pa.Table.from_pandas(df, schema=schema)
pa.parquet.write_table(table, "orders.parquet")
```

Particionamiento Hive-style

Parquet soporta particionamiento al estilo Hive, donde cada partición es un directorio:

```
# Escribir particionado por año/mes
df["year"] = df["order_date"].dt.year
df["month"] = df["order_date"].dt.month

df.to_parquet(
    "data/bronze/orders/",
    partition_cols=["year", "month"],
    index=False
)

# Resultado:
# data/bronze/orders/year=2024/month=1/part-0.parquet
# data/bronze/orders/year=2024/month=2/part-1.parquet
# ...
```

Leer particiones

```
# Leer solo un mes específico
import pyarrow.parquet as pq

dataset = pq.ParquetDataset(
    "data/bronze/orders/",
    filters=[("year", "=", 2024), ("month", "=", 6)]
)

table = dataset.read()
df = table.to_pandas()
print(f"Pedidos de junio 2024: {len(df)}")
```

Avro

Avro es un formato row-oriented, ideal para datos secuenciales y streaming.

```
import fastavro

schema = {
    "type": "record",
    "name": "Order",
    "fields": [
        {"name": "order_id", "type": "string"},
        {"name": "customer_id", "type": "int"},
        {"name": "total", "type": "float"},
        {"name": "status", "type": "string"},
    ]
}

records = [
    {"order_id": "ORD-001", "customer_id": 1, "total": 150.0, "status": "completed"},
    {"order_id": "ORD-002", "customer_id": 2, "total": 89.99, "status": "completed"},
]

with open("orders.avro", "wb") as f:
    fastavro.writer(f, schema, records)
```

¿Cuándo usar cada uno?

Formato	Úsalo cuando...
Parquet	Almacenes analíticos, data lakes, BI
Avro	Streaming, Kafka, transferencia entre sistemas
CSV	Exportaciones rápidas, interoperabilidad legacy
JSON	APIs, logs, datos semiestructurados

DuckDB: el SQLite del data engineering

DuckDB es una base de datos SQL embebida, optimizada para analítica. Ideal para pipelines:

```
import duckdb

# Consultar Parquet directamente
df = duckdb.sql("""
    SELECT
        strftime(order_date, '%Y-%m') AS mes,
        COUNT(*) AS pedidos,
        ROUND(SUM(total), 2) AS ingresos
    FROM 'data/bronze/orders/*.parquet'
    GROUP BY mes
    ORDER BY mes
""").df()

print(df)
```


Ejercicios

1. Convierte `orders` de TechStore de CSV a Parquet. Compara tamaños
2. Lee un Parquet con `pyarrow` usando solo las columnas `order_id` y `total`
3. Crea un dataset Parquet particionado por año/mes desde SQLite
4. Escribe un archivo Avro con 10 registros de `customers`
5. ¿Cuánto más pequeño es Parquet vs CSV para los datos de TechStore?
6. ¿Por qué Parquet es mejor que CSV para consultas analíticas?
7. Usa DuckDB para consultar archivos Parquet directamente
8. ¿Qué es el particionamiento Hive-style y por qué es útil?
9. Lee un Parquet de `order_items` filtrando por `quantity > 5`
10. ¿En qué escenario elegirías Avro en lugar de Parquet?

Capítulo 4: Data Lakes y Object Storage

¿Qué es un Data Lake?

Un **Data Lake** es un repositorio centralizado que almacena datos en su formato nativo (crudo), sin necesidad de schema previo.

Data Warehouse:

Solo datos estructurados, schema-on-write, caro

Data Lake:

Todos los formatos, schema-on-read, barato (object storage)

Data Lake vs Data Warehouse

Característica	Data Warehouse	Data Lake
Datos	Estructurados	Todos (estructurados, semi, no estructurados)
Schema	On-write	On-read
Formato	Tablas SQL	Archivos (Parquet, Avro, CSV, JSON)
Almacenamiento	Costoso (\$/GB)	Barato (object storage)
Uso	BI, reportes	Data Science, ML, pipelines
Mutabilidad	UPDATE/DELETE	Inmutable (nuevas versiones)

Object Storage

El object storage es la base de los data lakes modernos:

AWS S3 → El estándar de la industria

GCS → Google Cloud Storage

Azure → Azure Data Lake Storage (ADLS)

MinIO → S3-compatible, corre localmente (Docker)

Conceptos clave

Bucket → Contenedor lógico (como un "disco")

└─ key (ruta del objeto): "bronze/orders/orders_2024-01-01.parquet"

└─ metadata: tipo de contenido, fecha, etiquetas

└─ versiones: cada cambio crea una nueva versión

MinIO — S3 local con Docker

MinIO es un servidor de object storage compatible con la API de S3. Corre en Docker y es perfecto para desarrollo local.

Levantar MinIO

```
# docker-compose.yml
services:
  minio:
    image: minio/minio:latest
    ports:
      - "9000:9000" # API S3
      - "9001:9001" # Consola web
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin123
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data

volumes:
  minio_data:
```

```
docker compose up -d
# Consola: http://localhost:9001 (user: minioadmin / pass: minioadmin123)
```

Python + MinIO (boto3/S3 API)

```
import boto3
from botocore.config import Config

# Cliente S3 apuntando a MinIO local
s3_client = boto3.client(
    "s3",
    endpoint_url="http://localhost:9000",
    aws_access_key_id="minioadmin",
    aws_secret_access_key="minioadmin123",
    config=Config(signature_version="s3v4"),
    region_name="us-east-1",
)

# Crear bucket
s3_client.create_bucket(Bucket="techstore-bronze")

# Subir archivo
s3_client.upload_file(
    "data/bronze/orders/orders_2024-01-01.parquet",
    "techstore-bronze",
    "orders/year=2024/month=1/orders_2024-01-01.parquet"
)

# Listar objetos
response = s3_client.list_objects_v2(
```

```

        Bucket="techstore-bronze",
        Prefix="orders/year=2024/"
    )
    for obj in response.get("Contents", []):
        print(f"    {obj['Key']} ({obj['Size']} bytes)")

    # Descargar archivo
    s3_client.download_file(
        "techstore-bronze",
        "orders/year=2024/month=1/orders_2024-01-01.parquet",
        "data/temp/orders_backup.parquet"
    )

```

Estructura del Data Lake de TechStore

```

techstore-bronze/          # Datos crudos
├─ orders/year=2024/month=1/orders_2024-01-01.parquet
├─ orders/year=2024/month=1/orders_2024-01-02.parquet
├─ customers/customers_full.parquet
└─ products/products_full.parquet

techstore-silver/          # Datos limpios
├─ orders/year=2024/month=1/
├─ customers/
└─ products/

techstore-gold/            # Datos listos para negocio
├─ fact_orders/
├─ dim_customers/
└─ dim_products/

```

Script de carga a MinIO

```

# pipelines/load_to_minio.py
import boto3
from pathlib import Path
import logging
from config.loader import load_config

logger = logging.getLogger("load_to_minio")

def get_s3_client(config: dict):
    return boto3.client(
        "s3",
        endpoint_url=config["data_lake"]["endpoint"],
        aws_access_key_id=config["data_lake"]["access_key"],
        aws_secret_access_key=config["data_lake"]["secret_key"],
        config=Config(signature_version="s3v4"),
    )

```

```
def upload_parquet_to_bronze(local_path: Path, bucket: str, s3_key: str):
    client = get_s3_client(load_config())
    client.upload_file(str(local_path), bucket, s3_key)
    logger.info(f"Subido: {local_path} → s3://{bucket}/{s3_key}")

# Uso
upload_parquet_to_bronze(
    local_path=Path("data/bronze/orders/orders_2024-01-01.parquet"),
    bucket="techstore-bronze",
    s3_key="orders/year=2024/month=1/orders_2024-01-01.parquet"
)
```

Schema-on-read

El data lake no impone schema al escribir. Lo defines al leer:

```
import pyarrow.parquet as pq

# Schema implícito (desde el archivo)
table = pq.read_table("s3://techstore-bronze/orders/year=2024/month=1/")

# Schema explícito (forzar tipos)
from pyarrow import parquet as pq, Table
import pyarrow as pa

schema = pa.schema([
    ("order_id", pa.string()),
    ("total", pa.float64()),
    ("status", pa.string()),
])

table = pq.read_table(
    "s3://techstore-bronze/orders/year=2024/month=1/",
    schema=schema # Solo lee estas columnas
)
```

Ejercicios

1. Levanta MinIO con Docker y accede a la consola en `http://localhost:9001`
2. Crea los buckets `techstore-bronze`, `techstore-silver`, `techstore-gold`
3. Extrae `orders` de SQLite, conviértelo a Parquet y súbelo a MinIO (bronze)
4. Lista los objetos en `techstore-bronze/` desde Python
5. ¿Cuál es la diferencia entre un bucket y un directorio en MinIO?
6. Descarga un archivo de MinIO y verifica su integridad
7. ¿Qué ventaja tiene schema-on-read frente a schema-on-write?
8. Crea una función `list\_bronze\_tables()` que liste todas las tablas en bronze
9. Sube `customers` y `products` como Parquet a bronze con partición por fecha
10. ¿Por qué se usa `signature\_version="s4v4"` en la configuración de boto3?

Checkpoint 1: Data Lake de TechStore

Objetivo

Construir el data lake de TechStore con MinIO, aplicando los conceptos de Python para data engineering, formatos de datos (Parquet) y object storage.

Requisitos

- Docker Desktop instalado
- Python 3.10+ con `boto3`, `pyarrow`, `pandas`, `pyyaml`, `sqlite3`
- Archivo SQLite de TechStore (`techstore.db`)

Paso 1: Levantar MinIO

```
# docker-compose.yml del capítulo 4
docker compose up -d
# Verificar: http://localhost:9001
```

Paso 2: Estructura del proyecto

```
techstore_data_lake/
├── pipelines/
│   ├── __init__.py
│   ├── extract.py
│   ├── load_to_minio.py
│   └── utils.py
├── config/
│   └── config.yaml
├── docker-compose.yml
├── requirements.txt
└── .env
```

Paso 3: Configuración

```
# config/config.yaml
database:
  path: data/techstore.db
```

```
data_lake:
  endpoint: http://localhost:9000
  bucket_bronze: techstore-bronze
  bucket_silver: techstore-silver
  bucket_gold: techstore-gold
  access_key: ${MINIO_ACCESS_KEY}
  secret_key: ${MINIO_SECRET_KEY}
```

```
# .env
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin123
```

Paso 4: Script de extracción

```
# pipelines/extract.py
import sqlite3
import pandas as pd
from pathlib import Path
from datetime import date
from config.loader import load_config

def extract_table(table: str, output_dir: Path, dt: date = None) -> Path:
    config = load_config()
    db_path = config["database"]["path"]

    conn = sqlite3.connect(db_path)
    df = pd.read_sql_query(f"SELECT * FROM {table}", conn)
    conn.close()

    if dt:
        path = output_dir / f"{table}_{dt.isoformat()}.parquet"
    else:
        path = output_dir / f"{table}_full.parquet"

    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(path, index=False)
    return path
```

Paso 5: Script de carga

```
# pipelines/load_to_minio.py
import boto3
from botocore.config import Config
from pathlib import Path

def upload_file(local_path: Path, bucket: str, s3_key: str):
    client = boto3.client(
        "s3",
        endpoint_url="http://localhost:9000",
        aws_access_key_id="minioadmin",
```

```

        aws_secret_access_key="minioadmin123",
        config=Config(signature_version="s3v4"),
    )
    client.upload_file(str(local_path), bucket, s3_key)

def create_bucket(bucket: str):
    client = boto3.client(
        "s3",
        endpoint_url="http://localhost:9000",
        aws_access_key_id="minioadmin",
        aws_secret_access_key="minioadmin123",
        config=Config(signature_version="s3v4"),
    )
    client.create_bucket(Bucket=bucket)

```

Paso 6: Pipeline completo

```

# run_pipeline.py
from pipelines.extract import extract_table
from pipelines.load_to_minio import upload_file, create_bucket
from config.loader import load_config
from datetime import date
from pathlib import Path

def main():
    config = load_config()
    bronze = config["data_lake"]["bucket_bronze"]

    # Crear buckets si no existen
    for bucket in [bronze, "techstore-silver", "techstore-gold"]:
        create_bucket(bucket)

    # Extraer tablas principales
    today = date.today()
    for table in ["orders", "customers", "products", "order_items"]:
        local_path = extract_table(table, Path("data/extract"), today)
        s3_key = f"{table}/year={today.year}/month={today.month}/{local_path.name}"
        upload_file(local_path, bronze, s3_key)
        print(f"    {table} → s3://{bronze}/{s3_key}")

if __name__ == "__main__":
    main()

```

Entregables del Checkpoint 1

- [] MinIO corriendo con Docker
- [] Buckets creados (bronze, silver, gold)
- [] Script de extracción de tablas SQLite a Parquet
- [] Script de carga a MinIO

- [] Pipeline completo que extrae y carga tablas a bronze
- [] Datos accesibles desde la consola MinIO
- [] Logging configurado en los scripts

Preguntas de reflexión

1. ¿Por qué es mejor tener 3 buckets (bronze/silver/gold) que uno solo?
 2. ¿Qué ventaja tiene particionar por año/mes en el data lake?
 3. ¿Cómo cambiarías el pipeline para manejar tablas con millones de filas?
 4. ¿Qué información incluirías en los logs para debugging?
- ¡Felicidades! Tienes tu primer data lake funcional. En el Proyecto 2 lo transformarás con dbt.

Capítulo 5: Introducción a dbt

¿Qué es dbt?

dbt (data build tool) es una herramienta que permite a analistas e ingenieros transformar datos en su warehouse usando SQL, con buenas prácticas de software: testing, documentación, versionado y CI/CD.

```
Fuente: Tablas raw → dbt models (SQL) → Tablas transformadas
(SQLite, Parquet)      SELECT ...          (listas para BI)
```

dbt NO extrae ni carga datos. dbt SOLO transforma (la T de ELT).

dbt Core vs dbt Cloud

Característica	dbt Core	dbt Cloud
Coste	Gratuito (open source)	Desde ~\$100/mes
Instalación	pip install	SaaS
Entorno	CLI, local	Web + CLI
CI/CD	Manual	Integrado
Documentación	dbt docs generate	Hosteada
Ideal para	Aprendizaje, equipos pequeños	Equipos, empresas

En este libro usamos **dbt Core** (gratuito, local).

Instalación

```
pip install dbt-core dbt-duckdb
# dbt-duckdb: permite usar DuckDB como backend (perfecto para local)
```

Crear un proyecto dbt

```
dbt init techstore_dbt
cd techstore_dbt
```

Estructura generada:

```
techstore_dbt/
├─ models/
│   └─ example/
│       └─ my_first_dbt_model.sql
```

```
|
├── my_second_dbt_model.sql
├── analyses/
├── tests/
├── seeds/
├── macros/
├── snapshots/
├── dbt_project.yml
└── profiles.yml          # Configuración de conexión
```

Configuración de perfiles

```
# profiles.yml
techstore_dbt:
  outputs:
    dev:
      type: duckdb
      path: /data/techstore_warehouse.duckdb
      schema: main
      threads: 4
  target: dev
```

Primer modelo dbt

dbt transforma modelos SQL en tablas o vistas en tu base de datos.

```
-- models/staging/stg_orders.sql
-- Este modelo limpia orders: elimina nulos, estandariza fechas
SELECT
  order_id,
  customer_id,
  CAST(order_date AS DATE) AS order_date,
  ROUND(total, 2) AS total,
  COALESCE(status, 'unknown') AS status
FROM raw_orders
WHERE total > 0
```

Materializaciones

dbt puede materializar modelos de 4 formas:

Materialización	Descripción	Ideal para
`view`	Vista SQL (default)	Transformaciones ligeras
`table`	Tabla física recreada cada run	Datos de tamaño medio
`incremental`	Solo inserta/actualiza nuevos registros	Grandes volúmenes
`ephemeral`	CTE reutilizable, no se materializa	Lógica compartida

```
-- models/staging/stg_orders.sql
{{ config(materialized='view') }}

SELECT ... FROM raw_orders
```

```
-- models/marts/fact_orders.sql
{{ config(materialized='table') }}

SELECT ... FROM {{ ref('stg_orders') }}
```

dbt_project.yml

```
# dbt_project.yml
name: 'techstore_dbt'
version: '1.0.0'
config-version: 2

profile: 'techstore_dbt'

model-paths: ["models"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]

models:
  techstore_dbt:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts
```

Comandos básicos

```
# Ejecutar todos los modelos
dbt run

# Ejecutar solo un modelo específico
dbt run --select stg_orders

# Ejecutar modelos y sus dependencias
dbt run --select stg_orders+

# Compilar SQL sin ejecutar
dbt compile

# Generar documentación
dbt docs generate
dbt docs serve      # Abre servidor web con docs
```

Sources y ref

```
-- models/staging/stg_orders.sql
-- Usar source() en lugar de hardcodear nombres de tabla
```

```
{{ config(materialized='view') }}
```

```
SELECT
    order_id,
    customer_id,
    CAST(order_date AS DATE) AS order_date,
    ROUND(total, 2) AS total,
    COALESCE(status, 'unknown') AS status
FROM {{ source('techstore', 'orders') }}
```

```
WHERE total > 0
```

```
# models/sources.yml
version: 2

sources:
  - name: techstore
    database: main
    schema: main
    tables:
      - name: orders
      - name: customers
      - name: products
      - name: order_items
```

```
-- models/marts/fact_orders.sql
{{ config(materialized='table') }}
```

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.total,
    o.status
FROM {{ ref('stg_orders') }} o
```

Lineage (línea de datos)

dbt genera automáticamente el **lineage** de tus modelos:

```
stg_orders → fact_orders → dashboard_ventas
stg_customers → dim_customers → dashboard_ventas
                                     |
                                     ▼
                                reporte_ejecutivo
```

```
dbt docs generate
dbt docs serve
# Abre http://localhost:8080 y haz clic en "View lineage graph"
```

Ejercicios

1. Instala dbt-core y dbt-duckdb. Inicializa un proyecto ``techstore_dbt``
2. Crea un modelo ``stg_customers`` que limpie nombres y emails
3. Configura ``profiles.yml`` para usar DuckDB como warehouse local
4. Define una source ``techstore`` con las 4 tablas principales
5. Crea un modelo ``stg_order_items`` que calcule ``line_total = quantity * unit_price``
6. Ejecuta ``dbt run`` y verifica que se crearon las vistas
7. ¿Qué diferencia hay entre ``source()`` y ``ref()`` en dbt?
8. Genera la documentación con ``dbt docs generate`` y explora el lineage
9. ¿Por qué es útil tener staging models separados de marts models?
10. ¿Qué materialización usarías para una tabla de 10M filas que se actualiza diario?

Capítulo 6: Modelado en dbt — Sources, Ref y Materializaciones

El flujo de trabajo en dbt

Sources (tablas raw) → Staging (limpieza) → Intermediate (lógica) → Marts (negocio)

dbt fomenta un pipeline modular:

1. **Sources:** define las tablas de origen
2. **Staging:** modelos 1:1 con las fuentes — limpieza ligera
3. **Intermediate:** lógica de negocio, joins, agregaciones
4. **Marts:** tablas listas para BI (hechos y dimensiones)

Staging models

Un staging model por cada tabla fuente. Limpieza mínima:

```
-- models/staging/stg_products.sql
{{ config(materialized='view') }}

SELECT
    product_id,
    TRIM(name) AS product_name,
    category,
    ROUND(unit_price, 2) AS unit_price,
    COALESCE(stock, 0) AS stock
FROM {{ source('techstore', 'products') }}
```

Staging conventions

```
-- models/staging/stg_customers.sql
SELECT
    customer_id,
    TRIM(name) AS customer_name,
    LOWER(TRIM(email)) AS email,
    TRIM(city) AS city,
    TRIM(country) AS country,
    CAST(registration_date AS DATE) AS registration_date
FROM {{ source('techstore', 'customers') }}
```

Intermediate models

Lógica de negocio y joins entre staging models:

```
-- models/intermediate/int_order_details.sql
{{ config(materialized='view') }}
```

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.total,
    o.status,
    oi.product_id,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS line_total
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_order_items') }} oi ON o.order_id = oi.order_id
```

```
-- models/intermediate/int_customer_metrics.sql
{{ config(materialized='view') }}
```

```
SELECT
    o.customer_id,
    COUNT(DISTINCT o.order_id) AS total_pedidos,
    SUM(o.total) AS total_ingresos,
    AVG(o.total) AS ticket_promedio,
    MIN(o.order_date) AS primera_compra,
    MAX(o.order_date) AS ultima_compra,
    JULIANDAY(MAX(o.order_date)) - JULIANDAY(MIN(o.order_date)) AS dias_entre_compras
FROM {{ ref('stg_orders') }} o
GROUP BY o.customer_id
```

Marts models (Hechos y Dimensiones)

Tabla de hechos

```
-- models/marts/fact_orders.sql
{{ config(materialized='table') }}
```

```
SELECT
    order_id,
    customer_id,
    order_date,
    total,
    status,
    COUNT(DISTINCT product_id) AS productos_distintos,
    SUM(quantity) AS total_unidades,
    SUM(line_total) AS ingresos_brutos
FROM {{ ref('int_order_details') }}
GROUP BY 1, 2, 3, 4, 5
```


Dimensiones

```
-- models/marts/dim_customers.sql
{{ config(materialized='table') }}

SELECT
    c.customer_id,
    c.customer_name,
    c.email,
    c.city,
    c.country,
    c.registration_date,
    COALESCE(m.total_pedidos, 0) AS total_pedidos,
    COALESCE(m.total_ingresos, 0) AS total_ingresos,
    COALESCE(m.ticket_promedio, 0) AS ticket_promedio
FROM {{ ref('stg_customers') }} c
LEFT JOIN {{ ref('int_customer_metrics') }} m ON c.customer_id = m.customer_id
```

```
-- models/marts/dim_products.sql
{{ config(materialized='table') }}

SELECT
    product_id,
    product_name,
    category,
    unit_price,
    stock
FROM {{ ref('stg_products') }}
```

Mart analítico

```
-- models/marts/report_sales_overview.sql
{{ config(materialized='table') }}

SELECT
    DATE_TRUNC('month', o.order_date) AS mes,
    COUNT(DISTINCT o.order_id) AS pedidos,
    COUNT(DISTINCT o.customer_id) AS clientes_activos,
    SUM(o.total) AS ingresos,
    AVG(o.total) AS ticket_promedio,
    SUM(o.total_unidades) AS unidades_vendidas
FROM {{ ref('fact_orders') }} o
GROUP BY mes
ORDER BY mes
```

Modelos incrementales

Para tablas grandes que se actualizan periódicamente:

```
-- models/marts/fact_orders_incremental.sql
{{ config(
    materialized='incremental',
```

```

        unique_key='order_id',
        incremental_strategy='merge'
    ) }}

SELECT
    order_id,
    customer_id,
    order_date,
    total,
    status
FROM {{ ref('stg_orders') }}

{% if is_incremental() %}
    WHERE order_date > (SELECT MAX(order_date) FROM {{ this }})
{% endif %}

```

Dependencias y orden de ejecución

dbt resuelve el orden automáticamente según el lineage:

```

1. stg_orders, stg_customers, stg_products, stg_order_items
2. int_order_details, int_customer_metrics
3. fact_orders, dim_customers, dim_products
4. report_sales_overview

```

```

# Ver el DAG de dependencias
dbt run --select stg_orders+ # stg_orders + todo lo que depende de él

# Ejecutar solo marts
dbt run --select marts

```

Schema naming

dbt puede organizar modelos en schemas separados:

```

# dbt_project.yml
models:
  techstore_dbt:
    staging:
      +schema: staging
    intermediate:
      +schema: intermediate
    marts:
      +schema: marts

```

Ejercicios

1. Crea el staging model para `stg\_order\_items` con `line\_total` calculado
2. Crea un intermediate model `int\_product\_sales` que sume ventas por producto
3. Crea `dim\_date` como un modelo que genere una tabla de fechas (2020-2026)
4. ¿Qué ventaja tienen los modelos incrementales frente a tablas completas?

5. Crea ``fact_daily_orders`` que agregue pedidos por día
6. Configura schemas separados para staging, intermediate y marts
7. Ejecuta ``dbt run --select staging`` y verifica que solo corre staging
8. ¿Por qué es buena práctica tener staging models 1:1 con las fuentes?
9. Crea un mart ``report_top_products`` con top 10 productos por ingresos
10. Usa ``dbt run --select stg_orders+`` y observa qué modelos se ejecutan

Capítulo 7: Testing y Documentación en dbt

Por qué testear datos

Los datos tienen errores. Siempre. dbt te permite detectarlos automáticamente:

Errores típicos:

- Valores nulos en columnas que no deberían ser nulas
- IDs duplicados en tablas que deben ser únicas
- Valores fuera de rango (totales negativos, fechas futuras)
- Relaciones que no se cumplen (customer_id sin customer)

Schema tests (tests declarativos)

Los schema tests se definen en archivos YAML:

```
# models/schema.yml
version: 2

models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: total
        tests:
          - not_null
          - accepted_values:
              values: [ "> 0" ] # Test personalizado
      - name: status
        tests:
          - accepted_values:
              values: [ 'completed', 'pending', 'cancelled', 'refunded' ]
```

Tests integrados

```
# models/marts/schema.yml
version: 2
```

```

models:
  - name: fact_orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('dim_customers')
              field: customer_id
      - name: order_date
        tests:
          - not_null
          - dbt_utils.expression_is_true:
              expression: "order_date <= CURRENT_DATE"

  - name: dim_customers
    columns:
      - name: customer_id
        tests:
          - unique
          - not_null
      - name: email
        tests:
          - not_null
          - unique

```

Generic tests (tests parametrizables)

Puedes crear tus propios tests reutilizables:

```

-- tests/generic/test_positive_values.sql
{% test positive_values(model, column_name) %}

SELECT *
FROM {{ model }}
WHERE {{ column_name }} <= 0

{% endtest %}

```

```

# models/schema.yml
models:
  - name: stg_orders
    columns:
      - name: total
        tests:
          - positive_values

```

Otros tests genéricos

```
-- tests/generic/test_date_in_past.sql
{% test date_in_past(model, column_name) %}

SELECT *
FROM {{ model }}
WHERE {{ column_name }} > CURRENT_DATE

{% endtest %}
```

```
# models/schema.yml
- name: fact_orders
  columns:
    - name: order_date
      tests:
        - date_in_past
```

Data tests (tests SQL personalizados)

Para lógica más compleja:

```
-- tests/assert_order_total_is_correct.sql
-- Verifica que el total de orders = suma de line_totals en order_items
SELECT
  o.order_id,
  o.total AS order_total,
  SUM(oi.quantity * oi.unit_price) AS calculated_total
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_order_items') }} oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total
HAVING ABS(o.total - SUM(oi.quantity * oi.unit_price)) > 0.01
```

```
-- tests/assert_no_future_orders.sql
SELECT order_id, order_date
FROM {{ ref('fact_orders') }}
WHERE order_date > CURRENT_DATE
```

Ejecutar tests

```
# Ejecutar todos los tests
dbt test

# Tests de un modelo específico
dbt test --select stg_orders

# Tests solo de un tipo
dbt test --select test_type:unique

# Ver resultados
dbt test --select stg_orders --store_failures
```

Almacenar fallos

```
dbt test --store-failures
# Crea tablas con los registros que fallaron
# Útil para debugging y auditoría
```

Documentación en dbt

dbt genera documentación automática a partir de descripciones en YAML:

```
# models/marts/schema.yml
version: 2

models:
  - name: fact_orders
    description: "Tabla de hechos de pedidos. Cada fila es un pedido con métricas agregadas."
    columns:
      - name: order_id
        description: "Identificador único del pedido (PK)"
        tests:
          - unique
          - not_null
      - name: total
        description: "Monto total del pedido en USD. Incluye impuestos."
      - name: status
        description: "Estado del pedido: completed, pending, cancelled, refunded"

  - name: dim_customers
    description: "Dimensión de clientes con métricas de comportamiento."
    columns:
      - name: customer_id
        description: "Identificador único del cliente (PK)"
      - name: total_pedidos
        description: "Número total de pedidos realizados por el cliente"
      - name: ticket_promedio
        description: "Gasto promedio por pedido del cliente"
```

Generar y servir documentación

```
dbt docs generate
dbt docs serve
# Abre http://localhost:8080
# Incluye: descripciones, columnas, tests, lineage, SQL de cada modelo
```

dbt Project Evaluator

dbt tiene un paquete que evalúa la calidad de tu proyecto:

```
# packages.yml
packages:
  - package: dbt-labs/dbt_project_evaluator
    version: ">=0.1.0"
```

```
dbt deps      # Instalar paquetes
dbt run --select package:dbt_project_evaluator
dbt test --select package:dbt_project_evaluator
```

Buenas prácticas de testing

1. **Siempre testea PKs:** `unique` + `not\_null` en cada staging model
2. **Testea relaciones:** cada FK debe tener un test `relationships`
3. **Testea rangos:** fechas futuras, valores negativos, outliers
4. **Documenta mientras construyes:** la documentación es parte del código
5. **Falla rápido:** corre tests en cada `dbt run`

Ejercicios

1. Agrega tests `unique` y `not\_null` a `order\_id` en `stg\_orders`
2. Crea un test `relationships` entre `stg\_orders.customer\_id` y `stg\_customers.customer\_id`
3. Escribe un data test que verifique que no hay pedidos con total negativo
4. Crea un test genérico `not\_empty\_string` para columnas de texto
5. Documenta 3 modelos con descripciones y columnas
6. Genera la documentación con `dbt docs generate`
7. Ejecuta `dbt test` y corrige cualquier fallo
8. ¿Por qué es importante testear relaciones entre tablas?
9. Crea un test que verifique que `order\_items.quantity > 0`
10. ¿Qué ventaja tiene `dbt test --store-failures`?

Capítulo 8: dbt Avanzado — Jinja, Macros y Paquetes

Jinja en dbt

dbt usa **Jinja** (templating engine de Python) para hacer tus modelos SQL dinámicos y reutilizables.

```
-- Sin Jinja: hay que escribir cada columna manualmente
SELECT
    order_id,
    customer_id,
    order_date,
    total,
    status
FROM raw_orders
WHERE order_date >= '2024-01-01' AND order_date < '2024-02-01'

-- Con Jinja: parámetros y bucles
{% set start_date = '2024-01-01' %}
{% set end_date = '2024-02-01' %}

SELECT
    {% for col in ['order_id', 'customer_id', 'order_date', 'total', 'status'] %}
    {{ col }}{% if not loop.last %},{% endif %}
    {% endfor %}
FROM raw_orders
WHERE order_date >= '{{ start_date }}' AND order_date < '{{ end_date }}'
```

Variables de configuración

```
-- dbt_project.yml
vars:
    start_date: '2024-01-01'
    end_date: '2024-12-31'
    revenue_target: 1000000
```

```
-- models/marts/report_dashboard.sql
SELECT
    DATE_TRUNC('month', order_date) AS mes,
```

```

COUNT(*) AS pedidos,
SUM(total) AS ingresos,
SUM(total) / {{ var('revenue_target') }} * 100 AS pct_cumplimiento
FROM {{ ref('fact_orders') }}
WHERE order_date >= '{{ var('start_date') }}'
      AND order_date <= '{{ var('end_date') }}'
GROUP BY mes

```

Macros (funciones reutilizables)

```

-- macros/dates.sql
{% macro date_range_sql(start_date, end_date) %}
    WHERE order_date >= '{{ start_date }}' AND order_date <= '{{ end_date }}'
{% endmacro %}

-- Uso
SELECT ... FROM {{ ref('fact_orders') }} {{ date_range_sql('2024-01-01', '2024-06-30') }}

```

Macro para columnas comunes

```

-- macros/columns.sql
{% macro generate_surrogate_key(columns) %}
    MD5(CONCAT(
        {% for col in columns %}
            COALESCE(CAST({{ col }} AS VARCHAR), ''){% if not loop.last %} || '-' || {% endif %}
        {% endfor %}
    ))
{% endmacro %}

-- Uso
SELECT
    {{ generate_surrogate_key(['order_id', 'product_id']) }} AS order_item_sk,
    ...
FROM {{ ref('int_order_details') }}

```

Macro de logging

```

-- macros/logging.sql
{% macro log_info(message) %}
    {{ log("[INFO] " ~ message, info=True) }}
{% endmacro %}

{% macro log_warning(message) %}
    {{ log("[WARN] " ~ message, info=True) }}
{% endmacro %}

```

Hooks (eventos)

Los hooks ejecutan SQL antes o después de modelos:

```
# dbt_project.yml
```

```
models:
  techstore_dbt:
    +pre-hook:
      - "DELETE FROM {{ this }} WHERE 1=0" # Para cargas completas antes de insert
    +post-hook:
      - "INSERT INTO audit_log (model, run_at, rows_affected) VALUES ('{{ this.name }}', CURRENT_TIMESTAMP"
```

Paquetes (dbt packages)

Los paquetes son bibliotecas reutilizables de macros y modelos:

```
# packages.yml
packages:
  - package: dbt-labs/dbt_utils
    version: ">=1.0.0"
  - package: dbt-labs/codegen
    version: ">=0.12.0"
  - package: calogica/dbt_expectations
    version: ">=0.8.0"
```

```
dbt deps # Instalar paquetes
```

Ejemplos con dbt_utils

```
-- dbt_utils: pivot table
SELECT *
FROM {{ dbt_utils.pivot('status',
  dbt_utils.get_column_values(ref('stg_orders'), 'status'),
  agg='COUNT',
  prefix='orders_'
) }}
```

```
-- Equivalente a PIVOT en SQL dinámico
```

```
-- dbt_utils: surrogate key
SELECT
  {{ dbt_utils.generate_surrogate_key(['customer_id', 'order_date']) }} AS customer_day_sk,
  ...
```

Codegen: generar modelos automáticamente

```
# Generar staging models desde una fuente existente
dbt run-operation generate_source --args '{"schema_name": "main", "database_name": "main"}'

# Generar documentación base
dbt run-operation generate_docs
```

dbt Exposures

Los **exposures** documentan cómo se usan tus modelos downstream (BI tools, dashboards):

```
# models/exposures.yml
version: 2

exposures:
  - name: dashboard_ventas
    type: dashboard
    url: https://app.powerbi.com/...
    description: "Dashboard de ventas ejecutivo en Power BI"
    depends_on:
      - ref('fact_orders')
      - ref('dim_customers')
      - ref('report_sales_overview')
    owner:
      name: Alex Goyzueta
      email: alex@techstore.com

  - name: reporte_mensual
    type: analysis
    url: https://lookerstudio.google.com/...
    description: "Reporte mensual de KPIs"
    depends_on:
      - ref('report_sales_overview')
```

dbt Snapshot (SCD Tipo 2)

Los snapshots capturan cambios históricos en dimensiones lentas (SCD2):

```
-- snapshots/customers_snapshot.sql
{% snapshot customers_snapshot %}

{{
  config(
    target_schema='snapshots',
    unique_key='customer_id',
    strategy='check',
    check_cols=['email', 'city', 'country'],
    invalidate_hard_deletes=True,
  )
}}

SELECT * FROM {{ source('techstore', 'customers') }}
```

```
dbt snapshot # Ejecuta los snapshots
```

Buenas prácticas avanzadas

1. Usa ``dbt_utils`` para no reinventar la rueda
2. **Macros para lógica repetitiva:** no copies SQL, crea macros

3. **Variables para configuraciones:** fechas, umbrales, targets
4. **Hooks para auditoría:** registra cuándo se ejecutó cada modelo
5. **Snapshots para SCD2:** solo cuando necesites historia de cambios
6. **Exposures para documentar uso downstream**

Ejercicios

1. Crea una macro ``generate_timestamp_key`` que genere una SK con timestamp + rand
2. Usa ``dbt_utils.surrogate_key`` para crear una SK en ``fact_orders``
3. Instala el paquete ``dbt_utils`` y usa ``dbt_utils.date_spine`` para generar fechas
4. Crea un hook que registre en una tabla ``audit_log`` cada ejecución
5. Define una variable ``revenue_target`` y úsala en un modelo
6. Crea una exposure para tu dashboard Power BI
7. Usa ``codegen`` para generar un staging model automáticamente
8. ¿Qué ventaja tiene usar macros frente a SQL repetitivo?
9. Crea un snapshot SCD2 para ``products``
10. ¿Por qué los packages son importantes en dbt?

Checkpoint 2: Pipeline dbt de TechStore

Objetivo

Construir un pipeline completo de transformación con dbt que tome los datos del data lake (bronze) y los transforme en un modelo analítico listo para BI (gold).

Requisitos

- dbt-core + dbt-duckdb instalados (Capítulo 5)
- Data Lake con datos en bronze (Checkpoint 1)
- DuckDB como warehouse local

Paso 1: Configurar proyecto dbt

```
dbt init techstore_dbt
cd techstore_dbt
```

```
# profiles.yml
techstore_dbt:
  outputs:
    dev:
      type: duckdb
      path: /data/techstore_gold.duckdb
      schema: main
      threads: 4
      target: dev
```

Paso 2: Definir sources

```
# models/sources.yml
version: 2

sources:
  - name: bronze
    database: main
    schema: main
    tables:
      - name: orders
      - name: customers
```

- name: products
- name: order_items

Los datos se cargan desde Parquet usando DuckDB:

```
-- models/staging/stg_orders.sql
{{ config(materialized='view') }}

SELECT *
FROM read_parquet('data/bronze/orders/*.parquet')
```

Paso 3: Staging models

```
-- models/staging/stg_customers.sql
{{ config(materialized='view') }}

SELECT
    customer_id,
    TRIM(name) AS customer_name,
    LOWER(TRIM(email)) AS email,
    TRIM(city) AS city,
    TRIM(country) AS country,
    CAST(registration_date AS DATE) AS registration_date
FROM read_parquet('data/bronze/customers/*.parquet')
```

```
-- models/staging/stg_products.sql
{{ config(materialized='view') }}

SELECT
    product_id,
    TRIM(name) AS product_name,
    category,
    ROUND(unit_price, 2) AS unit_price,
    COALESCE(stock, 0) AS stock
FROM read_parquet('data/bronze/products/*.parquet')
```

```
-- models/staging/stg_order_items.sql
{{ config(materialized='view') }}

SELECT
    order_id,
    product_id,
    quantity,
    unit_price,
    (quantity * unit_price) AS line_total
FROM read_parquet('data/bronze/order_items/*.parquet')
```

Paso 4: Intermediate models

```
-- models/intermediate/int_order_details.sql
{{ config(materialized='view') }}
```

```

SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.total,
    o.status,
    oi.product_id,
    oi.quantity,
    oi.unit_price,
    oi.line_total
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_order_items') }} oi ON o.order_id = oi.order_id

```

Paso 5: Marts models (Gold)

```

-- models/marts/fact_orders.sql
{{ config(materialized='table') }}

SELECT
    order_id,
    customer_id,
    order_date,
    total,
    status,
    COUNT(DISTINCT product_id) AS productos_distintos,
    SUM(quantity) AS total_unidades,
    SUM(line_total) AS ingresos_brutos
FROM {{ ref('int_order_details') }}
GROUP BY 1, 2, 3, 4, 5

```

```

-- models/marts/dim_customers.sql
{{ config(materialized='table') }}

SELECT * FROM {{ ref('stg_customers') }}

```

```

-- models/marts/report_sales_daily.sql
{{ config(materialized='table') }}

SELECT
    order_date,
    COUNT(DISTINCT order_id) AS pedidos,
    COUNT(DISTINCT customer_id) AS clientes_activos,
    SUM(total) AS ingresos,
    AVG(total) AS ticket_promedio
FROM {{ ref('fact_orders') }}
GROUP BY order_date
ORDER BY order_date

```

Paso 6: Tests


```
# models/schema.yml
version: 2

models:
  - name: stg_orders
    description: "Orders desde el data lake bronze"
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: total
        tests: [not_null]
      - name: status
        tests:
          - accepted_values:
              values: ['completed', 'pending', 'cancelled', 'refunded']

  - name: fact_orders
    description: "Hechos de pedidos agregados"
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('dim_customers')
              field: customer_id
```

Paso 7: Ejecutar

```
# Instalar paquetes (opcional)
dbt deps

# Ejecutar todos los modelos
dbt run

# Ejecutar tests
dbt test

# Generar documentación
dbt docs generate
dbt docs serve
```

Entregables del Checkpoint 2

- [] Proyecto dbt configurado con profiles.yml
- [] Sources definidas para datos bronze
- [] 4 staging models (orders, customers, products, order_items)
- [] 1 intermediate model (int_order_details)

- [] 3 marts models (fact_orders, dim_customers, report_sales_daily)
- [] Tests para PKs, FKs y valores aceptados
- [] Documentación generada (dbt docs serve)
- [] duckdb generado con datos gold

Preguntas de reflexión

1. ¿Qué capa de la medallion architecture corresponde a marts? ¿Por qué?
2. ¿Cómo cambiaría el pipeline si tuviéramos 50 tablas fuente en lugar de 4?
3. ¿Qué tests adicionales agregarías para garantizar calidad de datos?
4. ¿Cómo conectarías estos datos gold a Power BI o Looker Studio?

Capítulo 9: Introducción a Apache Airflow

¿Qué es Apache Airflow?

Airflow es una plataforma open source para programar, orquestar y monitorear workflows (pipelines). Creada por Airbnb en 2014, ahora es el estándar de la industria.

Sin Airflow:

- Scripts Python en cron jobs (difíciles de monitorear)
- Fallos silenciosos (nadie sabe que falló)
- Dependencias manuales entre scripts

Con Airflow:

- DAGs visuales con dependencias explícitas
- Logs centralizados, alertas automáticas
- Reintentos, backfills, scheduling flexible

Conceptos clave

DAG (Directed Acyclic Graph)

Un **DAG** es un conjunto de tareas con dependencias. "Acíclico" significa que no puede haber loops (tarea A B A).

```
extract → transform → load → done
```

Task

Cada paso del pipeline es un **task** (operador individual en el DAG).

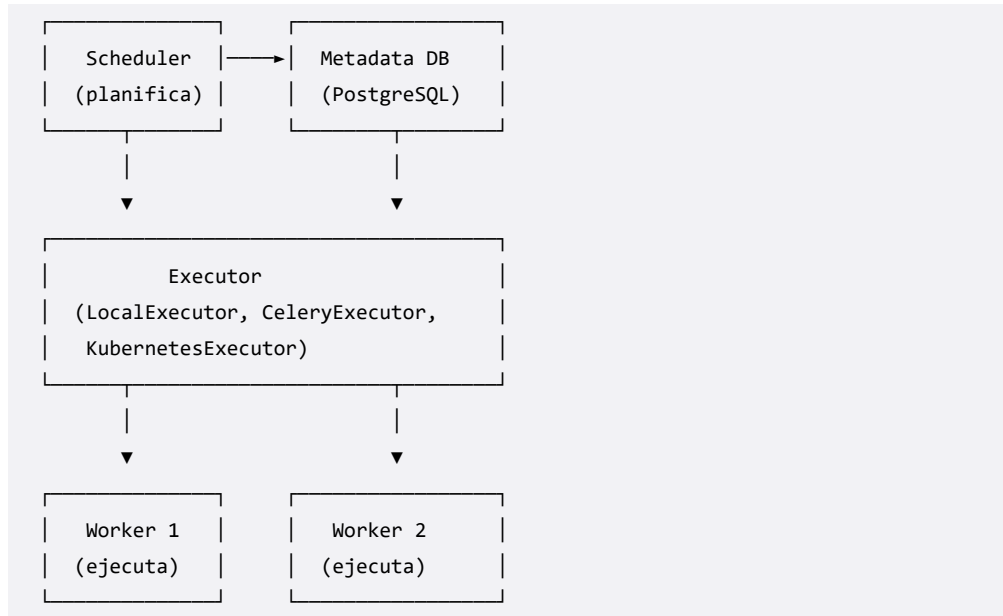
Operator

Define QUÉ hace cada task:

Operator	Propósito
<code>`PythonOperator`</code>	Ejecuta una función Python
<code>`BashOperator`</code>	Ejecuta un comando bash
<code>`PostgresOperator`</code>	Ejecuta SQL en PostgreSQL
<code>`DockerOperator`</code>	Corre un contenedor Docker
<code>`S3FileTransformOperator`</code>	Transforma archivos en S3

`EmailOperator`	Envía un email
`BranchPythonOperator`	Decide qué camino seguir condicionalmente
`Sensor`	Espera a que ocurra algo (archivo, API, SQL)

Arquitectura de Airflow



Para desarrollo local: `LocalExecutor` + `SQLite` (suficiente).

Levantar Airflow con Docker

```

# docker-compose-airflow.yml
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres_data:/var/lib/postgresql/data

  airflow-webserver:
    image: apache/airflow:2.10.0
    ports:
      - "8080:8080"
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    volumes:
      - ./dags:/opt/airflow/dags
      - ./data:/opt/airflow/data
    depends_on:
      - postgres
  
```

```

command: >
    bash -c "airflow db init && airflow users create
    --username admin --password admin --firstname Admin
    --lastname Admin --role Admin --email admin@techstore.com
    && airflow webserver"

airflow-scheduler:
    image: apache/airflow:2.10.0
    environment:
        AIRFLOW__CORE__EXECUTOR: LocalExecutor
        AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    volumes:
        - ./dags:/opt/airflow/dags
        - ./data:/opt/airflow/data
    depends_on:
        - postgres
    command: airflow scheduler

volumes:
    postgres_data:

```

Tu primer DAG

```

# dags/01_hello_techstore.py
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "techstore",
    "depends_on_past": False,
    "email_on_failure": True,
    "email": ["alerts@techstore.com"],
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="techstore_hello",
    description="Primer DAG de TechStore",
    default_args=default_args,
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["techstore", "example"],
) as dag:

    start = BashOperator(
        task_id="start",
        bash_command="echo 'Iniciando pipeline TechStore'",

```

```

)

def extract_data():
    import sqlite3, pandas as pd
    conn = sqlite3.connect("/opt/airflow/data/techstore.db")
    df = pd.read_sql_query("SELECT COUNT(*) AS total FROM orders", conn)
    conn.close()
    print(f"Total orders: {df['total'].iloc[0]}")
    return int(df["total"].iloc[0])

extract = PythonOperator(
    task_id="extract_orders_count",
    python_callable=extract_data,
)

end = BashOperator(
    task_id="end",
    bash_command="echo 'Pipeline completado'",
)

start >> extract >> end

```

Schedule en Airflow

```

# Opciones de schedule
schedule="@daily"           # Diario a medianoche
schedule="@hourly"          # Cada hora
schedule="@weekly"          # Domingo a medianoche
schedule="@monthly"         # 1 del mes a medianoche
schedule="0 6 * * *"       # Cron: 6:00 AM todos los días
schedule="0 8 * * 1-5"     # 8:00 AM lunes a viernes
schedule="0 0 1 * *"       # 1 de cada mes
schedule=None               # Manual (solo trigger)

```

Variables de Airflow

```

# CLI
airflow variables set TECHSTORE_DB_PATH /opt/airflow/data/techstore.db
airflow variables set MINIO_ENDPOINT http://minio:9000

```

```

from airflow.models import Variable

db_path = Variable.get("TECHSTORE_DB_PATH")
minio_url = Variable.get("MINIO_ENDPOINT")

```

Conexiones

```

# CLI
airflow connections add 'techstore_db' \

```

```
--conn-type 'sqlite' \  
--conn-host '/opt/airflow/data/techstore.db'  
  
airflow connections add 'minio_s3' \  
--conn-type 'aws' \  
--conn-login 'minioadmin' \  
--conn-password 'minioadmin123' \  
--conn-extra '{"endpoint_url": "http://minio:9000"}'
```

Monitoreo básico

Desde la UI de Airflow (`http://localhost:8080`):

DAGs: Lista de todos los DAGs
Graph View: Dependencias visuales
Tree View: Historial de ejecuciones
Gantt: Duración de cada task
Logs: Output de cada task
Code: Código fuente del DAG

Ejercicios

1. Levanta Airflow con Docker (webserver + scheduler + postgres)
2. Crea un DAG simple con 3 tasks: ``start process_data end``
3. Programa el DAG para ejecutarse diariamente a las 7:00 AM
4. Agrega reintentos: 3 intentos con 2 minutos de espera
5. Usa ``Variable.get`` para leer la ruta de la base de datos SQLite
6. ¿Qué diferencia hay entre un DAG y un Task en Airflow?
7. Crea un ``PythonOperator`` que cuente las filas de ``customers``
8. ¿Para qué sirve ``catchup=False`` en un DAG?
9. Explora la UI de Airflow (Graph View, Tree View, Logs)
10. Crea un DAG que se ejecute solo manualmente (sin schedule)

Capítulo 10: Airflow Avanzado — TaskFlow, XComs y Sensores

TaskFlow API (Airflow 2.0+)

La **TaskFlow API** simplifica la creación de DAGs usando decoradores Python:

```
from airflow.decorators import dag, task
from datetime import datetime, timedelta

@dag(
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["techstore"],
)
def techstore_pipeline():

    @task
    def extract_orders():
        import sqlite3, pandas as pd
        conn = sqlite3.connect("/opt/airflow/data/techstore.db")
        df = pd.read_sql_query("SELECT * FROM orders", conn)
        conn.close()
        return df.to_dict("records") # JSON-serializable

    @task
    def transform_orders(orders: list):
        import pandas as pd
        df = pd.DataFrame(orders)
        df["total"] = df["total"].round(2)
        df["order_date"] = pd.to_datetime(df["order_date"]).dt.date
        return df.to_dict("records")

    @task
    def load_orders(orders: list):
        import pandas as pd
        df = pd.DataFrame(orders)
        df.to_parquet("/opt/airflow/data/silver/orders_clean.parquet", index=False)
        print(f"Cargados {len(df)} pedidos")
        return len(df)
```



```

# Flujo implícito
orders = extract_orders()
clean = transform_orders(orders)
count = load_orders(clean)

dag = techstore_pipeline()

```

XComs — Compartir datos entre tasks

Los **XComs** permiten pasar mensajes entre tasks:

```

@task
def extract():
    import sqlite3
    conn = sqlite3.connect("/opt/airflow/data/techstore.db")
    count = conn.execute("SELECT COUNT(*) FROM orders").fetchone()[0]
    conn.close()
    return {"table": "orders", "rows": count} # Se guarda automáticamente en XCom

@task
def process(metadata: dict):
    # metadata llega desde XCom automáticamente (TaskFlow)
    print(f"Procesando {metadata['table']}: {metadata['rows']} filas")
    return metadata["rows"]

@task
def notify(row_count: int):
    from airflow.utils.email import send_email
    # Enviar notificación
    print(f"Pipeline completado: {row_count} filas procesadas")

```

XComs tradicional (sin TaskFlow)

```

from airflow.operators.python import PythonOperator

def push_xcom(ti):
    ti.xcom_push(key="order_count", value=1234)

def pull_xcom(ti):
    count = ti.xcom_pull(key="order_count", task_ids="extract")
    print(f"Orders: {count}")

extract = PythonOperator(task_id="extract", python_callable=push_xcom)
process = PythonOperator(task_id="process", python_callable=pull_xcom)
extract >> process

```

BranchPythonOperator — Condicionales

```

from airflow.operators.python import BranchPythonOperator

```

```
def decide_branch(**kwargs):
    import sqlite3
    conn = sqlite3.connect("/opt/airflow/data/techstore.db")
    count = conn.execute("SELECT COUNT(*) FROM orders").fetchone()[0]
    conn.close()

    if count == 0:
        return "skip_pipeline"
    return "run_pipeline"

with DAG(...) as dag:
    check = BranchPythonOperator(
        task_id="check_data",
        python_callable=decide_branch,
    )

    skip = BashOperator(task_id="skip_pipeline", bash_command="echo 'Sin datos'")
    run = BashOperator(task_id="run_pipeline", bash_command="echo 'Ejecutando...'")
    end = BashOperator(task_id="end", bash_command="echo 'Fin'", trigger_rule="none_failed_or_skipped")

    check >> [skip, run] >> end
```

Sensores — Esperar condiciones

Los **sensores** son tasks que esperan hasta que se cumple una condición:

```
from airflow.sensors.filesystem import FileSensor
from airflow.sensors.sql import SqlSensor
from airflow.sensors.time_delta import TimeDeltaSensor
from airflow.sensors.external_task import ExternalTaskSensor

# Esperar a que llegue un archivo
wait_for_file = FileSensor(
    task_id="wait_for_orders_file",
    filepath="/opt/airflow/data/raw/orders_new.parquet",
    fs_conn_id="fs_default",
    poke_interval=30, # Revisar cada 30 segundos
    timeout=3600,     # Timeout después de 1 hora
    mode="reschedule", # Libera el slot mientras espera
)

# Esperar a que una tabla tenga datos
wait_for_data = SqlSensor(
    task_id="wait_for_orders",
    conn_id="techstore_db",
    sql="SELECT COUNT(*) FROM orders WHERE order_date = CURRENT_DATE",
    success=lambda row: row[0] > 0,
    poke_interval=60,
    timeout=7200,
)
```

Sensor personalizado

```

from airflow.sensors.base import BaseSensorOperator

class MinIOSensor(BaseSensorOperator):
    def __init__(self, bucket: str, key: str, minio_conn_id: str, **kwargs):
        super().__init__(**kwargs)
        self.bucket = bucket
        self.key = key
        self.minio_conn_id = minio_conn_id

    def poke(self, context):
        import boto3
        client = boto3.client("s3",
                               endpoint_url="http://minio:9000",
                               aws_access_key_id="minioadmin",
                               aws_secret_access_key="minioadmin123",
                               )
        try:
            client.head_object(Bucket=self.bucket, Key=self.key)
            self.log.info(f"Archivo encontrado: s3://{self.bucket}/{self.key}")
            return True
        except:
            self.log.info(f"Esperando archivo: s3://{self.bucket}/{self.key}")
            return False

# Uso
wait_minio = MinIOSensor(
    task_id="wait_orders_bronze",
    bucket="techstore-bronze",
    key="orders/year=2024/month=1/orders_2024-01-01.parquet",
    poke_interval=30,
    timeout=3600,
)

```

Task Groups — Organizar tareas

```

from airflow.utils.task_group import TaskGroup

with DAG(...) as dag:
    start = BashOperator(task_id="start", bash_command="echo 'Inicio'")

    with TaskGroup(group_id="extract_all") as extract_group:
        extract_orders = BashOperator(task_id="orders", bash_command="echo 'Orders'")
        extract_customers = BashOperator(task_id="customers", bash_command="echo 'Customers'")
        extract_products = BashOperator(task_id="products", bash_command="echo 'Products'")

    with TaskGroup(group_id="load_all") as load_group:
        load_bronze = BashOperator(task_id="bronze", bash_command="echo 'Cargando bronze'")
        load_silver = BashOperator(task_id="silver", bash_command="echo 'Cargando silver'")

    end = BashOperator(task_id="end", bash_command="echo 'Fin'")

```

```
start >> extract_group >> load_group >> end
```

Trigger Rules

Controlan cuándo se ejecuta un task según el estado de sus dependencias:

```
from airflow.utils.trigger_rule import TriggerRule

end = BashOperator(
    task_id="end",
    bash_command="echo 'Fin'",
    trigger_rule=TriggerRule.ALL_DONE,      # Corre incluso si falla
    # trigger_rule="all_success"           # Default: solo si todo OK
    # trigger_rule="one_success"           # Si al menos uno OK
    # trigger_rule="one_failed"            # Si al menos uno falló
    # trigger_rule="none_failed"           # Ninguno falló
    # trigger_rule="none_failed_or_skipped" # Ninguno falló o fue skipeado
    # trigger_rule="dummy"                 # Siempre, sin dependencias
)
```

Backfill — Reprocesar históricos

```
# CLI
airflow dags backfill techstore_pipeline \
    --start-date 2024-01-01 \
    --end-date 2024-06-30

# O desde la UI: DAG → Trigger DAG w/ config → {"start_date": "2024-01-01", "end_date": "2024-06-30"}
```

Ejercicios

1. Convierte tu DAG anterior a TaskFlow API con decoradores
2. Crea un sensor que espere a que exista un archivo Parquet en MinIO
3. Usa XComs para pasar el número de pedidos extraídos entre tasks
4. Implementa un BranchPythonOperator que decida si ejecutar la pipeline según la fecha
5. Crea un TaskGroup para "extract_all_tables" con 4 tasks paralelos
6. ¿Qué diferencia hay entre `mode="poke"` y `mode="reschedule"` en un sensor?
7. Usa `trigger\_rule="all\_done"` para un task de limpieza que siempre se ejecute
8. Ejecuta un backfill para reprocesar enero 2024
9. Crea un sensor personalizado que verifique un registro en SQL
10. ¿Por qué los XComs deben contener datos serializables (JSON)?

Capítulo 11: Integración Airflow + dbt

El tándem perfecto

dbt transforma datos. Airflow orquesta cuándo y cómo. Juntos forman el corazón del stack moderno de data engineering:

```
Airflow
├─ extract_from_sqlite → MinIO (bronze)
├─ trigger_dbt_run → dbt (bronze → silver → gold)
└─ notify_slack → Alerta de éxito/fallo
```

Opción 1: BashOperator + dbt CLI

```
from airflow.operators.bash import BashOperator

run_dbt = BashOperator(
    task_id="dbt_run",
    bash_command="cd /opt/airflow/dbt/techstore_dbt && dbt run",
    env={
        "DBT_PROFILES_DIR": "/opt/airflow/dbt/techstore_dbt",
        "DBT_ENV": "dev",
    },
)
```

DAG completo

```
from airflow.decorators import dag, task
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

@dag(
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["techstore", "dbt"],
)
def techstore_dbt_pipeline():

    # 1. Extraer datos a bronze
    @task
    def extract_to_bronze():
```

```

import sqlite3, pandas as pd
import boto3

tables = ["orders", "customers", "products", "order_items"]
s3 = boto3.client("s3", endpoint_url="http://minio:9000",
                  aws_access_key_id="minioadmin",
                  aws_secret_access_key="minioadmin123")

conn = sqlite3.connect("/opt/airflow/data/techstore.db")
for table in tables:
    df = pd.read_sql_query(f"SELECT * FROM {table}", conn)
    local_path = f"/opt/airflow/data/bronze/{table}.parquet"
    df.to_parquet(local_path, index=False)
    s3.upload_file(local_path, "techstore-bronze", f"{table}/{table}_latest.parquet")
conn.close()

# 2. Ejecutar dbt
dbt_run = BashOperator(
    task_id="dbt_run",
    bash_command="cd /opt/airflow/dbt/techstore_dbt && dbt run --target dev",
)

# 3. Testear dbt
dbt_test = BashOperator(
    task_id="dbt_test",
    bash_command="cd /opt/airflow/dbt/techstore_dbt && dbt test",
)

# 4. Generar documentación
dbt_docs = BashOperator(
    task_id="dbt_docs",
    bash_command="cd /opt/airflow/dbt/techstore_dbt && dbt docs generate",
)

extract_to_bronze() >> dbt_run >> dbt_test >> dbt_docs

dag = techstore_dbt_pipeline()

```

Opción 2: DockerOperator para aislar dbt

```

from airflow.providers.docker.operators.docker import DockerOperator

dbt_run = DockerOperator(
    task_id="dbt_run",
    image="ghcr.io/dbt-labs/dbt-duckdb:latest",
    command="run",
    api_version="auto",
    auto_remove=True,
    docker_url="unix://var/run/docker.sock",
    network_mode="bridge",
    mounts=[

```

```

        Mount(source="/opt/airflow/dbt/techstore_dbt",
              target="/usr/app", type="bind"),
        Mount(source="/opt/airflow/data",
              target="/data", type="bind"),
    ],
    environment={
        "DBT_PROFILES_DIR": "/usr/app",
    },
)

```

Opción 3: dbt Cloud + Airflow

```

from airflow.providers.http.operators.http import SimpleHttpOperator

trigger_dbt_cloud_job = SimpleHttpOperator(
    task_id="trigger_dbt_cloud_job",
    http_conn_id="dbt_cloud",
    endpoint="api/v2/accounts/{account_id}/runs/",
    method="POST",
    data=json.dumps({"job_definition_id": 12345}),
    headers={"Content-Type": "application/json", "Authorization": "Bearer YOUR_TOKEN"},
)

check_dbt_cloud_run = SimpleHttpOperator(
    task_id="check_dbt_cloud_run",
    http_conn_id="dbt_cloud",
    endpoint="api/v2/accounts/{account_id}/runs/{run_id}/",
    method="GET",
)

```

Pipeline completo: Extract dbt Notify

```

@dag(
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["techstore"],
)

def techstore_full_pipeline():

    @task
    def extract():
        # Extraer datos a bronze (Parquet en MinIO)
        return {"status": "ok", "tables": ["orders", "customers"]}

    @task
    def run_dbt():
        # Ejecutar dbt run + dbt test
        import subprocess
        result = subprocess.run(

```

```

        ["dbt", "run"],
        cwd="/opt/airflow/dbt/techstore_dbt",
        capture_output=True, text=True
    )
    if result.returncode != 0:
        raise Exception(f"dbt failed: {result.stderr}")
    return {"dbt_run": "success"}

@task
def notify_slack(status: dict):
    from slack_sdk import WebClient
    client = WebClient(token="xoxb-your-token")
    client.chat_postMessage(
        channel="#data-pipelines",
        text=f"Pipeline TechStore completado: {status}"
    )

extract_status = extract()
dbt_status = run_dbt()
notify_slack(dbt_status)

dag = techstore_full_pipeline()

```

Buenas prácticas Airflow + dbt

1. **Entornos separados:** dev, staging, prod (cada uno con su target dbt)
2. **Variables de entorno:** no hardcodees rutas ni credenciales
3. **Logging:** configura `dbt run --log-format JSON` para logs estructurados
4. **Alertas:** Slack/Email cuando dbt falle (con el log de error)
5. **Idempotencia:** dbt debe poder ejecutarse múltiples veces sin duplicar datos
6. **Backfills:** Airflow permite reprocesar fechas históricas fácilmente

Ejercicios

1. Crea un DAG que ejecute `dbt run` usando BashOperator
2. Agrega un task `dbt\_test` después de `dbt\_run`
3. Usa `DockerOperator` para ejecutar dbt en un contenedor aislado
4. Integra una notificación Slack/email cuando dbt falle
5. ¿Qué ventaja tiene separar extract, dbt y notify en tasks distintos?
6. Crea un DAG con TaskFlow que ejecute dbt y capture el log
7. ¿Por qué es importante la idempotencia en pipelines de datos?
8. Configura un DAG que ejecute dbt solo si el extract fue exitoso
9. ¿Cómo manejarías un fallo en dbt test? ¿Detendrías el pipeline?
10. Diseña un DAG que ejecute 3 modelos dbt en paralelo

Capítulo 12: Monitoreo, Alertas y SLAs

¿Por qué monitorear?

Los pipelines fallan. Es cuestión de cuándo, no de si. Sin monitoreo:

```
Día 1: Pipeline falla a las 3 AM
Día 5: Alguien nota que el dashboard no se actualizó
Día 6: Se descubre el fallo
Día 7: Se repara
→ 7 días sin datos actualizados
```

Con monitoreo:

```
3:00 AM: Pipeline falla
3:01 AM: Alerta en Slack
3:15 AM: Ingeniero recibe alerta en el móvil
3:30 AM: Pipeline reparado y reprocesado
→ 30 minutos de retraso
```

Alertas por email en Airflow

Configuración nativa de Airflow:

```
default_args = {
    "owner": "techstore",
    "email": ["data-eng@techstore.com", "alerts@techstore.com"],
    "email_on_failure": True,      # Email cuando un task falla
    "email_on_retry": True,       # Email cuando reintenta
    "email_on_success": False,    # No enviar email en éxito
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}
```

```
# Configurar SMTP en airflow.cfg
[smtp]
smtp_host = smtp.gmail.com
smtp_starttls = True
smtp_ssl = False
smtp_user = data-eng@techstore.com
smtp_password = app-password-here
smtp_port = 587
smtp_mail_from = data-eng@techstore.com
```

Alertas en Slack

```
from airflow.providers.slack.operators.slack_webhook import SlackWebhookOperator

SLACK_CONN_ID = "slack_techstore"

def send_slack_alert(context):
    dag_id = context["dag"].dag_id
    task_id = context["task"].task_id
    execution_date = context["execution_date"]
    log_url = context["task_instance"].log_url

    message = f"""
    Pipeline Falló
    • DAG: {dag_id}
    • Task: {task_id}
    • Fecha: {execution_date}
    • Logs: {log_url}
    """

    SlackWebhookOperator(
        task_id="slack_alert",
        slack_webhook_conn_id=SLACK_CONN_ID,
        message=message,
    ).execute(context=context)
```

```
# Usar en DAG
with DAG(
    dag_id="techstore_pipeline",
    default_args=default_args,
    on_failure_callback=send_slack_alert, # Callback global
) as dag:
    ...
```

SLA (Service Level Agreements)

Los **SLAs** definen el tiempo máximo que un task puede tardar:

```
from airflow.operators.python import PythonOperator

extract = PythonOperator(
    task_id="extract_orders",
    python_callable=extract_orders,
    sla=timedelta(hours=2), # Debe completar en ≤ 2 horas
)

# Si el SLA se incumple:
# 1. Se envía email de SLA miss
# 2. Aparece en la UI con icono de alerta
# 3. Se registra en task_instance.sla_miss
```

Configurar SLA

```
# airflow.cfg
[email]
email_backend = airflow.utils.email.send_email_smtp

[sla]
sla_miss_callback = techstore_monitoring.sla_callback
default_sla = timedelta(hours=4)
```

Monitoreo con logs estructurados

```
import json
import logging

# Logger estructurado (JSON)
logger = logging.getLogger("techstore.pipeline")

def log_event(event_type: str, **kwargs):
    log_entry = {
        "event": event_type,
        "pipeline": "techstore",
        **kwargs
    }
    logger.info(json.dumps(log_entry))

# Uso
log_event("extract_complete", table="orders", rows=5678, duration_sec=12.3)
log_event("dbt_run", model_count=15, test_count=30, tests_passed=30)
log_event("pipeline_failed", error="Connection timeout", duration_sec=120)
```

Pool de recursos

Airflow permite limitar concurrencia con **Pools**:

```
# CLI
airflow pools set extract_pool 2 "Máximo 2 extract simultáneos"
airflow pools set dbt_pool 1 "Solo 1 dbt run a la vez"
```

```
extract = PythonOperator(
    task_id="extract_orders",
    python_callable=extract_orders,
    pool="extract_pool",
    pool_slots=1,
)
```

SLAs por DAG

```
from airflow.models.dag import DAG
from datetime import timedelta
```

```

dag = DAG(
    dag_id="techstore_sla_example",
    schedule="@daily",
    default_args={
        "sla": timedelta(hours=3), # SLA global
    },
    sla_miss_callback=slack_sla_miss, # Callback específico
)

```

Monitoreo externo

Health checks

```

# dags/health_check.py
from airflow.decorators import dag, task
from datetime import datetime

@dag(schedule="*/5 * * * *", start_date=datetime(2024, 1, 1), catchup=False)
def techstore_health_check():

    @task
    def check_minio():
        import boto3
        client = boto3.client("s3", endpoint_url="http://minio:9000",
                               aws_access_key_id="minioadmin",
                               aws_secret_access_key="minioadmin123")
        buckets = client.list_buckets()
        assert len(buckets["Buckets"]) > 0
        return {"minio": "ok"}

    @task
    def check_dbt():
        import subprocess
        result = subprocess.run(["dbt", "--version"], capture_output=True, text=True)
        assert result.returncode == 0
        return {"dbt": "ok"}

    @task
    def check_db():
        import sqlite3
        conn = sqlite3.connect("/opt/airflow/data/techstore.db")
        conn.execute("SELECT 1")
        conn.close()
        return {"sqlite": "ok"}

    [check_minio(), check_dbt(), check_db()]

dag = techstore_health_check()

```

Dashboard de monitoreo

Crea un DAG que genere métricas de pipeline:

```
@task
def collect_metrics():
    from airflow.models import DagRun, TaskInstance
    from airflow.utils.state import State

    # Estado de los últimos 30 DAG runs
    dag_runs = DagRun.find(dag_id="techstore_pipeline")
    successes = sum(1 for dr in dag_runs if dr.state == State.SUCCESS)
    failures = sum(1 for dr in dag_runs if dr.state == State.FAILED)

    return {
        "total_runs": len(dag_runs),
        "success_rate": successes / len(dag_runs) * 100,
        "failures": failures,
    }
```

Ejercicios

1. Configura alertas por email cuando un DAG falle
2. Crea un callback `on\_failure\_callback` que envíe una alerta a Slack
3. Define un SLA de 30 minutos para el task `dbt\_run`
4. Crea un health check DAG que verifique MinIO, dbt y SQLite cada 5 minutos
5. ¿Qué diferencia hay entre `retries` y `sla` en Airflow?
6. Implementa un logger estructurado (JSON) para el pipeline
7. Configura un Pool para limitar a 1 tarea de dbt simultánea
8. ¿Por qué es importante tener un DAG de health check separado?
9. Crea un task que colectione métricas semanales del pipeline
10. Diseña un flujo de alertas: fallo Slack Email Reprocesamiento automático

Checkpoint 3: Orquestación Completa

TechStore

Objetivo

Integrar Airflow y dbt en un pipeline completo: extraer datos desde SQLite a MinIO (bronze), transformarlos con dbt (silver gold), y generar documentación automática.

Requisitos

- Airflow funcionando (Docker)
- dbt-core + dbt-duckdb instalados en el contenedor Airflow
- Proyecto dbt de TechStore (Checkpoint 2)
- MinIO con datos en bronze (Checkpoint 1)

Paso 1: Estructura del proyecto Airflow

```
airflow/
├─ dags/
│   └─ techstore_complete.py  # DAG principal
├─ dbt/
│   └─ techstore_dbt/        # Proyecto dbt del Checkpoint 2
├─ data/
│   └─ techstore.db          # Base de datos fuente
├─ docker-compose.yml        # Airflow + Postgres + MinIO
└─ requirements.txt          # dbt-core, dbt-duckdb, boto3, pandas
```

Paso 2: DAG completo

```
# dags/techstore_complete.py
from airflow.decorators import dag, task
from airflow.operators.bash import BashOperator
from airflow.providers.slack.operators.slack_webhook import SlackWebhookOperator
from datetime import datetime, timedelta
import os

DBT_DIR = "/opt/airflow/dbt/techstore_dbt"
DATA_DIR = "/opt/airflow/data"
```

```

default_args = {
    "owner": "techstore",
    "email": ["data-eng@techstore.com"],
    "email_on_failure": True,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

@dag(
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    default_args=default_args,
    tags=["techstore", "production"],
)
def techstore_complete():

    # — Step 1: Extract —
    @task
    def extract_orders():
        import sqlite3, pandas as pd
        conn = sqlite3.connect(f"{DATA_DIR}/techstore.db")
        df = pd.read_sql_query("SELECT * FROM orders", conn)
        conn.close()
        path = f"{DATA_DIR}/bronze/orders.parquet"
        df.to_parquet(path, index=False)
        return {"table": "orders", "rows": len(df), "path": path}

    @task
    def extract_customers():
        import sqlite3, pandas as pd
        conn = sqlite3.connect(f"{DATA_DIR}/techstore.db")
        df = pd.read_sql_query("SELECT * FROM customers", conn)
        conn.close()
        path = f"{DATA_DIR}/bronze/customers.parquet"
        df.to_parquet(path, index=False)
        return {"table": "customers", "rows": len(df), "path": path}

    # — Step 2: dbt —
    dbt_run = BashOperator(
        task_id="dbt_run",
        bash_command=f"cd {DBT_DIR} && dbt run --target dev",
    )

    dbt_test = BashOperator(
        task_id="dbt_test",
        bash_command=f"cd {DBT_DIR} && dbt test",
    )

    dbt_docs = BashOperator(
        task_id="dbt_docs",
        bash_command=f"cd {DBT_DIR} && dbt docs generate",
    )

```

```

)

# — Step 3: Notify —
@task
def notify_success(extract_result: dict):
    print(f"Pipeline completado: {extract_result['table']} - {extract_result['rows']} filas")
    # Opcional: enviar a Slack
    return {"status": "success", "rows": extract_result["rows"]}

@task
def notify_failure():
    print("Pipeline falló - revisar logs")

# — Dependencies —
orders = extract_orders()
customers = extract_customers()

[orders, customers] >> dbt_run >> dbt_test >> dbt_docs
dbt_docs >> notify_success(orders)

dag = techstore_complete()

```

Paso 3: Docker Compose integrado

```

# docker-compose.yml
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres_data:/var/lib/postgresql/data

  minio:
    image: minio/minio:latest
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin123
    command: server /data --console-address ":9001"

  airflow-webserver:
    build:
      context: .
      dockerfile: Dockerfile.airflow
    ports:
      - "8080:8080"

```



```

environment:
  AIRFLOW__CORE__EXECUTOR: LocalExecutor
  AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
volumes:
  - ./dags:/opt/airflow/dags
  - ./dbt:/opt/airflow/dbt
  - ./data:/opt/airflow/data
depends_on:
  - postgres
  - minio
command: >
  bash -c "airflow db init && airflow users create
  --username admin --password admin --firstname Admin
  --lastname Admin --role Admin --email admin@techstore.com
  && airflow webserver"

airflow-scheduler:
  build:
    context: .
    dockerfile: Dockerfile.airflow
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
  volumes:
    - ./dags:/opt/airflow/dags
    - ./dbt:/opt/airflow/dbt
    - ./data:/opt/airflow/data
  depends_on:
    - postgres
  command: airflow scheduler

volumes:
  postgres_data:

```

```

# Dockerfile.airflow
FROM apache/airflow:2.10.0
RUN pip install dbt-core dbt-duckdb boto3 pandas pyarrow

```

Paso 4: Notificaciones

```

# dags/techstore_complete.py (agregar al final del DAG)
on_failure_callback = SlackWebhookOperator(
    task_id="slack_notify",
    slack_webhook_conn_id="slack_default",
    message=" Pipeline TechStore falló. Revisar Airflow.",
)

```

Entregables del Checkpoint 3

- [] Docker Compose con Airflow + Postgres + MinIO

- [] DAG `techstore\_complete` con extract dbt notify
- [] dbt run, dbt test, dbt docs generate como tasks separados
- [] Extract paralelo de orders y customers
- [] Notificación de éxito/fallo
- [] Pipeline ejecutándose correctamente en Airflow
- [] Documentación dbt generada automáticamente

Preguntas de reflexión

1. ¿Por qué separar extract, dbt run y dbt test en tasks distintos?
2. ¿Qué ventaja tiene el extract paralelo de orders y customers?
3. ¿Cómo modificarías el pipeline para manejar 50 tablas fuente?
4. ¿Qué métricas de performance monitorearías en este pipeline?

Capítulo 13: Docker para Data Engineering

¿Por qué Docker?

Los pipelines de datos dependen de múltiples servicios: bases de datos, object storage, orquestadores, herramientas de transformación. Docker permite empaquetar cada servicio en un contenedor reproducible.

```
Sin Docker:
    "En mi máquina funciona" → 3 días configurando dependencias

Con Docker:
    docker compose up → Todo funcionando en 30 segundos
```

Conceptos clave

```
Imagen → Plantilla (read-only)
    └─ docker build -t techstore-pipeline .

Contenedor → Instancia en ejecución de una imagen
    └─ docker run techstore-pipeline

Dockerfile → Receta para construir la imagen

Docker Compose → Orquestación de múltiples contenedores
```

Dockerfile para un pipeline Python

```
# Dockerfile.pipeline
FROM python:3.11-slim

WORKDIR /app

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*
```

```
# Copiar requirements e instalar dependencias Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código
COPY pipelines/ ./pipelines/
COPY config/ ./config/
COPY run_pipeline.py .

# Ejecutar pipeline
CMD ["python", "run_pipeline.py"]
```

```
# requirements.txt
pandas==2.2.0
pyarrow==15.0.0
boto3==1.34.0
pyyaml==6.0.1
sqlite3-helper==1.0.0
```

Build y ejecución

```
docker build -t techstore-pipeline -f Dockerfile.pipeline .
docker run --rm -v data:/app/data techstore-pipeline
```

Docker Compose para el stack TechStore

```
# docker-compose.yml
services:
  # — Data Lake —
  minio:
    image: minio/minio:latest
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin123
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 30s
      timeout: 10s
      retries: 3

  # — Base de datos fuente —
  sqlite:
    image: nouchka/sqlite3:latest
    volumes:
```

```

    - ./data:/data
    stdin_open: true
    tty: true

# — Pipeline de extracción —
extract:
  build:
    context: .
    dockerfile: Dockerfile.extract
  volumes:
    - ./data:/app/data
    - ./config:/app/config
  depends_on:
    minio:
      condition: service_healthy

# — dbt —
dbt:
  image: ghcr.io/dbt-labs/dbt-duckdb:latest
  volumes:
    - ./dbt:/usr/app
    - ./data:/data
  working_dir: /usr/app
  command: ["run"]

volumes:
  minio_data:

```

Docker networks

Los contenedores se comunican entre sí a través de redes Docker:

```

services:
  minio:
    networks:
      - techstore_net

  airflow:
    networks:
      - techstore_net

networks:
  techstore_net:
    driver: bridge

```

Dentro de la red, los contenedores se resuelven por nombre de servicio:

```

# Desde el contenedor extract
s3_client = boto3.client(
    "s3",
    endpoint_url="http://minio:9000", # Nombre del servicio, no localhost
    aws_access_key_id="minioadmin",
    aws_secret_access_key="minioadmin123",

```

)

Volúmenes y persistencia

```
services:
  minio:
    volumes:
      - minio_data:/data          # Volumen nombrado (persiste)
      - ./backup:/backup         # Bind mount (comparte con host)

volumes:
  minio_data:                    # Se crea automáticamente
```

Multi-stage build para imágenes más pequeñas

```
# Stage 1: Build
FROM python:3.11-slim AS builder
COPY requirements.txt .
RUN pip install --user --no-warn-script-location -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim
COPY --from=builder /root/.local /root/.local
COPY pipelines/ ./pipelines/
COPY run_pipeline.py .
ENV PATH=/root/.local/bin:$PATH
CMD ["python", "run_pipeline.py"]

# Imagen final: ~200 MB vs ~1 GB sin multi-stage
```

Buenas prácticas Docker para datos

1. **Imágenes pequeñas:** usa `python:3.11-slim`, no `python:3.11`
2. **Capas:** ordena las capas de menos a más cambiantes (requirements antes que código)
3. **.dockerignore:** excluye `\_\_pycache\_\_`, `.git`, `.env`, `data/`
4. **Healthchecks:** verifica que servicios como MinIO estén listos
5. **No correr como root:** usa `USER airflow` en imágenes de Airflow
6. **Logs a stdout:** los logs deben ir a stdout/stderr, no a archivos

```
# Buenas prácticas
FROM python:3.11-slim

RUN addgroup --system app && adduser --system --group app
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
USER app

CMD ["python", "run_pipeline.py"]
```

```
# .dockerignore
__pycache__/
*.pyc
.git/
.env
data/
*.md
tests/
```

Ejercicios

1. Crea un Dockerfile para un pipeline Python que extraiga datos de SQLite
2. Construye la imagen y ejecútala con `docker run`
3. Usa Docker Compose para levantar MinIO + tu pipeline
4. Configura un healthcheck para MinIO
5. ¿Por qué es importante el orden de las capas en un Dockerfile?
6. Crea un multi-stage build para reducir el tamaño de tu imagen
7. Usa un volumen para persistir los datos de MinIO
8. ¿Cómo se comunican dos contenedores en la misma red Docker?
9. Agrega `.dockerignore` a tu proyecto
10. ¿Qué ventaja tiene usar `python:3.11-slim` en lugar de `python:3.11`?

Capítulo 14: CI/CD para Datos

¿Por qué CI/CD en datos?

El código de datos (dbt models, pipelines Python, DAGs de Airflow) debería tratarse como software: versionado, testeado y desplegado automáticamente.

Sin CI/CD:

"Funciona en mi máquina" → push a main → todos los dashboards rotos

Con CI/CD:

push → tests automáticos → deploy → pipelines verificados

CI/CD vs Data CI/CD

Aspecto	CI/CD tradicional	CI/CD para datos
Testea	Código	Código + datos
Errores	Syntax, unit tests	Nulos, duplicados, freshness
Deploy	Nuevo código	Nuevos modelos + backfill
Rollback	git revert	dbt snapshot + rerun

GitHub Actions para dbt

```
# .github/workflows/dbt_ci.yml
name: dbt CI

on:
  pull_request:
    branches: [main]
    paths:
      - 'dbt/**/*.sql'
      - 'dbt/**/*.yaml'

jobs:
  dbt-test:
    runs-on: ubuntu-latest

    services:
      duckdb:
        image: ghcr.io/dbt-labs/dbt-duckdb:latest
```



```

steps:
  - uses: actions/checkout@v4

  - name: Install dbt
    run: pip install dbt-core dbt-duckdb

  - name: dbt deps
    run: dbt deps
    working-directory: dbt/techstore_dbt

  - name: dbt compile
    run: dbt compile
    working-directory: dbt/techstore_dbt

  - name: dbt test (schema only)
    run: dbt test --select test_type:singular
    working-directory: dbt/techstore_dbt

  - name: dbt docs generate
    run: dbt docs generate
    working-directory: dbt/techstore_dbt

  - name: Upload docs artifact
    uses: actions/upload-artifact@v4
    with:
      name: dbt-docs
      path: dbt/techstore_dbt/target/

```

GitHub Actions para pipelines Python

```

# .github/workflows/pipeline_ci.yml
name: Pipeline CI

on:
  push:
    branches: [develop, main]
    paths:
      - 'pipelines/**'
      - 'config/**'
      - 'requirements.txt'

jobs:
  test-pipeline:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:

```

```
python-version: '3.11'

- name: Install dependencies
  run: |
    pip install -r requirements.txt
    pip install pytest pytest-mock

- name: Run unit tests
  run: pytest tests/ -v

- name: Lint with Ruff
  uses: astral-sh/ruff-action@v1
  with:
    src: "./pipelines"

- name: Type check with mypy
  run: mypy pipelines/
```

Tests para pipelines

```
# tests/test_extract.py
import pytest
from pipelines.extract import extract_table
from pathlib import Path

def test_extract_orders_returns_parquet(tmp_path):
    """Extracción de orders debe generar un archivo Parquet válido."""
    result = extract_table("orders", tmp_path)
    assert result.exists()
    assert result.suffix == ".parquet"
    assert result.stat().st_size > 0

def test_extract_customers_columns(tmp_path):
    """Extracción de customers debe incluir columnas esperadas."""
    import pandas as pd
    result = extract_table("customers", tmp_path)
    df = pd.read_parquet(result)
    expected = {"customer_id", "name", "email", "city", "country"}
    assert expected.issubset(set(df.columns))

def test_extract_orders_no_negative_totals(tmp_path):
    """Ningún pedido debe tener total negativo."""
    import pandas as pd
    result = extract_table("orders", tmp_path)
    df = pd.read_parquet(result)
    assert (df["total"] >= 0).all()
```

CI/CD para DAGs de Airflow

```
# .github/workflows/airflow_ci.yml
```

```

name: Airflow DAG CI

on:
  pull_request:
    paths:
      - 'airflow/dags/**'

jobs:
  validate-dags:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install Airflow
        run: |
          pip install apache-airflow==2.10.0
          pip install -r airflow/requirements.txt

      - name: Initialize Airflow DB
        run: airflow db init

      - name: Validate DAGs
        run: airflow dags list --output yaml
        working-directory: airflow

      - name: Test DAG integrity
        run: python -c "
from airflow.models import DagBag
dagbag = DagBag(dag_folder='dags')
assert len(dagbag.import_errors) == 0, f'DAG errors: {dagbag.import_errors}'
print(f'DAGs válidos: {len(dagbag.dags)}')
"
        working-directory: airflow

```

dbt CI: pruebas en PRs

```

# .github/workflows/dbt_pr.yml
name: dbt PR Checks

on:
  pull_request:
    paths:
      - 'dbt/**'

jobs:

```

```

dbt-ci:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Install dbt
      run: pip install dbt-core dbt-duckdb

    - name: dbt debug
      run: dbt debug
      working-directory: dbt/techstore_dbt

    - name: dbt deps
      run: dbt deps
      working-directory: dbt/techstore_dbt

    - name: dbt compile
      run: dbt compile --target ci
      working-directory: dbt/techstore_dbt

    - name: dbt run (limited)
      run: dbt run --select staging --target ci
      working-directory: dbt/techstore_dbt

    - name: dbt test
      run: dbt test --target ci
      working-directory: dbt/techstore_dbt

    - name: Comment PR on failure
      if: failure()
      uses: actions/github-script@v7
      with:
        script: |
          github.rest.issues.createComment({
            issue_number: context.issue.number,
            owner: context.repo.owner,
            repo: context.repo.repo,
            body: ' dbt tests failed. Check run: ${github.server_url}/${github.repository}/actions/runs/${github.run_id}'
          })

```

Deploy automatizado

```

# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-pipeline:

```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4

  - name: Build Docker image
    run: docker build -t techstore-pipeline:latest .

  - name: Push to registry
    run: |
      docker tag techstore-pipeline:latest ghcr.io/${{ github.repository }}/pipeline:latest
      docker push ghcr.io/${{ github.repository }}/pipeline:latest

  - name: Deploy Airflow DAGs
    run: |
      rsync -avz airflow/dags/ airflow-server:/opt/airflow/dags/
      ssh airflow-server "airflow dags trigger techstore_complete"

```

Buenas prácticas de CI/CD para datos

1. **Testea en PRs:** no esperes a main para descubrir errores
2. **Entorno CI efímero:** usa DuckDB o SQLite en CI (no dependas de infraestructura real)
3. **Testea datos + código:** schema tests + unit tests + integration tests
4. **Cachea dependencias:** `pip install` con cache para builds más rápidos
5. **Comentarios automáticos:** dbt test falla comenta en el PR qué falló
6. **Deploy progresivo:** primero staging, luego producción

Ejercicios

1. Crea un workflow de GitHub Actions que ejecute `dbt compile` en cada PR
2. Agrega un paso que ejecute `dbt test` solo en modelos modificados
3. Escribe tests unitarios para tu pipeline `extract\_table`
4. Crea un workflow que valide que los DAGs de Airflow se importan correctamente
5. ¿Por qué es importante testear en PRs y no solo en main?
6. Implementa un step que comente el PR con el resultado de dbt test
7. ¿Qué ventaja tiene DuckDB como target de CI frente a una base real?
8. Configura Ruff (linter) en el workflow CI
9. Crea un deploy job que reconstruya la imagen Docker y la suba a un registry
10. ¿Cómo harías rollback de un modelo dbt desplegado con errores?

Capítulo 15: Proyecto Final — Data Platform TechStore

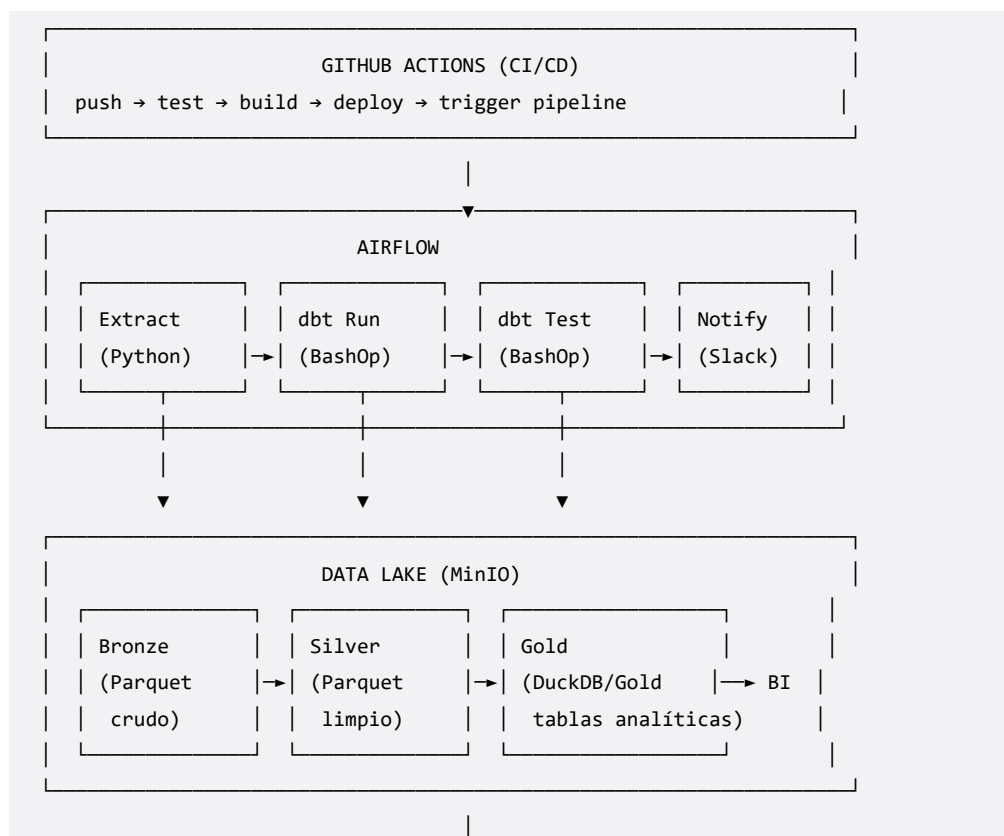
La visión completa

En los proyectos anteriores construiste:

1. **Data Lake** con MinIO y Parquet
2. **Transformaciones** con dbt (bronze silver gold)
3. **Orquestación** con Airflow
4. **Contenedores** con Docker
5. **CI/CD** con GitHub Actions

Ahora es momento de integrar todo en una **Data Platform completa** para TechStore.

Arquitectura final



▼

Power BI / Looker
(Dashboards)

Paso 1: Repositorio completo

```
techstore-data-platform/  
├─ .github/  
│   └─ workflows/  
│       ├── dbt_ci.yml  
│       ├── pipeline_ci.yml  
│       └─ deploy.yml  
├─ airflow/  
│   ├── dags/  
│   │   ├── techstore_complete.py  
│   │   ├── techstore_health.py  
│   │   └─ techstore_monitoring.py  
│   ├── Dockerfile.airflow  
│   └─ requirements.txt  
├─ dbt/  
│   └─ techstore_dbt/  
│       ├── models/  
│       │   ├── staging/  
│       │   ├── intermediate/  
│       │   └─ marts/  
│       ├── tests/  
│       ├── macros/  
│       ├── snapshots/  
│       ├── dbt_project.yml  
│       └─ profiles.yml  
├─ pipelines/  
│   ├── extract.py  
│   ├── load_to_minio.py  
│   └─ utils.py  
├─ config/  
│   └─ config.yaml  
├─ tests/  
│   └─ test_extract.py  
├─ docker-compose.yml  
├─ Dockerfile.pipeline  
├─ .env.example  
├─ .dockerignore  
├─ .gitignore  
└─ README.md
```

Paso 2: Configuración de variables

```
# .env (no commiteado)
```

```

MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=minioadmin123
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/xxx/yyy/zzz
DBT_PROFILES_DIR=./dbt/techstore_dbt
TECHSTORE_DB_PATH=./data/techstore.db

```

Paso 3: DAG de producción final

```

# airflow/dags/techstore_complete.py
from airflow.decorators import dag, task
from airflow.operators.bash import BashOperator
from airflow.providers.slack.operators.slack_webhook import SlackWebhookOperator
from datetime import datetime, timedelta
from airflow.models import Variable
import json

DBT_DIR = "/opt/airflow/dbt/techstore_dbt"
DATA_DIR = "/opt/airflow/data"
SLACK_CONN = "slack_default"

default_args = {
    "owner": "techstore",
    "email": ["data-eng@techstore.com"],
    "email_on_failure": True,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

@dag(
    schedule="0 6 * * *",          # 6:00 AM todos los días
    start_date=datetime(2024, 1, 1),
    catchup=False,
    default_args=default_args,
    tags=["techstore", "production"],
)
def techstore_production():

    # — Extract —
    @task
    def extract_all():
        import sqlite3, pandas as pd
        from pathlib import Path

        tables = ["orders", "customers", "products", "order_items"]
        bronze_dir = Path(f"{DATA_DIR}/bronze")
        bronze_dir.mkdir(parents=True, exist_ok=True)

        conn = sqlite3.connect(f"{DATA_DIR}/techstore.db")
        for table in tables:
            df = pd.read_sql_query(f"SELECT * FROM {table}", conn)
            path = bronze_dir / f"{table}.parquet"

```



```

        df.to_parquet(path, index=False)
    conn.close()
    return {"status": "ok", "tables": tables}

# — Load to MinIO —
@task
def load_to_bronze():
    import boto3
    from pathlib import Path

    s3 = boto3.client("s3",
        endpoint_url="http://minio:9000",
        aws_access_key_id="minioadmin",
        aws_secret_access_key="minioadmin123")

    bronze_dir = Path(f"{DATA_DIR}/bronze")
    for f in bronze_dir.glob("*.parquet"):
        s3.upload_file(str(f), "techstore-bronze", f.name)

# — dbt pipeline —
dbt_run = BashOperator(
    task_id="dbt_run",
    bash_command=f"cd {DBT_DIR} && dbt run --target prod",
    sla=timedelta(hours=1),
)

dbt_test = BashOperator(
    task_id="dbt_test",
    bash_command=f"cd {DBT_DIR} && dbt test",
)

dbt_docs = BashOperator(
    task_id="dbt_docs",
    bash_command=f"cd {DBT_DIR} && dbt docs generate",
)

# — Notifications —
@task
def notify_complete():
    print(" Pipeline TechStore completado exitosamente")

@task
def notify_failure():
    SlackWebhookOperator(
        task_id="slack_failure",
        slack_webhook_conn_id=SLACK_CONN,
        message=" Pipeline TechStore FALLÓ. Revisar Airflow.",
    ).execute(context={})

# — Pipeline flow —
extract_result = extract_all()
extract_result >> load_to_bronze() >> dbt_run >> dbt_test >> dbt_docs

```

```
dbt_docs >> notify_complete()

dag = techstore_production()
```

Paso 4: README del proyecto

```
# TechStore Data Platform

## Stack
- **Data Lake**: MinIO (S3-compatible) – Parquet
- **Transform**: dbt (DuckDB)
- **Orchestration**: Apache Airflow
- **Container**: Docker
- **CI/CD**: GitHub Actions

## Quick Start
```

1. Clone

```
git clone https://github.com/techstore/data-platform.git
```

2. Environment

```
cp .env.example .env
```

Edit .env with your values

3. Start services

`docker compose up -d`

4. Access

**Airflow: <http://localhost:8080>
(admin/admin)**

**MinIO: <http://localhost:9001>
(minioadmin/minioadmin123)**

5. Trigger pipeline

Airflow UI DAGs

techstore_production Trigger

```
## Pipeline
| Step | Tool | Description |
|-----|-----|-----|
| Extract | Python | SQLite → Parquet |
| Load | Python | Parquet → MinIO (bronze) |
| Transform | dbt | Bronze → Silver → Gold |
| Test | dbt | Schema + data tests |
| Docs | dbt | Auto-generated documentation |
| Notify | Slack | Success/failure alerts |

## Project Structure
(see tree above)
```

Paso 5: Runbook de operaciones

```
## Runbook: TechStore Data Platform

### Pipeline falló
1. Revisar Airflow → Logs del task que falló
2. Si es error de datos: corregir fuente, rerun
3. Si es error de infraestructura: verificar Docker, MinIO
4. Reprocess: `airflow dags backfill techstore_production -s YYYY-MM-DD`

### dbt test falló
1. Revisar `dbt test --store-failures` para ver registros específicos
2. Corregir modelo o datos fuente
3. `dbt run --select model_name && dbt test --select model_name`

### MinIO caído
1. `docker compose restart minio`
2. Verificar health: `curl http://localhost:9000/minio/health/live`
3. Reprocess data lake: `airflow dags trigger techstore_production`

### Deploy nuevo modelo dbt
1. Crear rama → PR → CI tests automáticos
2. Merge a main → deploy automático
```

Ejercicios

1. Escribe el README completo de tu Data Platform
2. Crea el docker-compose.yml final con todos los servicios
3. Configura variables de entorno en un `.env`` file
4. Crea un runbook con 3 escenarios de fallo comunes
5. ¿Qué métricas de salud monitorearías en producción?
6. Diseña un plan de disaster recovery para la plataforma
7. ¿Cómo escalarías el pipeline para 100 tablas fuente?
8. Prepara una presentación de 5 slides de la arquitectura
9. ¿Qué mejoras harías para el próximo release (v2.0)?
10. Documenta cómo un nuevo miembro del equipo empezaría a trabajar

Checkpoint 4: Data Platform Corporativa

Objetivo

Integrar todos los componentes en una **Data Platform corporativa** completa con CI/CD, documentación y runbook de operaciones.

Requisitos

- Docker Desktop
- Python 3.10+
- Git + GitHub (o GitHub alternativo)
- Proyectos de los Checkpoints 1-3 funcionando

Paso 1: Repositorio completo

```
mkdir -p techstore-data-platform && cd techstore-data-platform
git init
```

Copia y organiza los archivos de los checkpoints anteriores en la estructura del Capítulo 15.

Paso 2: Configurar CI/CD

```
# .github/workflows/ci.yml
name: TechStore Data Platform CI

on:
  pull_request:
    branches: [main]

jobs:
  test-extract:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
```

```

- run: pip install -r requirements.txt
- run: pytest tests/ -v

validate-dbt:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: pip install dbt-core dbt-duckdb
    - run: dbt deps && dbt compile
    working-directory: dbt/techstore_dbt

validate-airflow:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: pip install apache-airflow==2.10.0
    - run: airflow db init && python -c "
from airflow.models import DagBag
dag = DagBag(dag_folder='airflow/dags')
assert not dag.import_errors, f'Errors: {dag.import_errors}'
print(f'{len(dag.dags)} DAGs OK')
"

```

Paso 3: Ficha técnica del sistema

```

# Ficha Técnica – TechStore Data Platform

## Información General
- Sistema: TechStore Data Platform
- Versión: 1.0
- Fecha: 2026
- Responsable: [Tu nombre]

## Componentes
- Fuente: SQLite (techstore.db)
- Data Lake: MinIO (S3-compatible)
- Formato: Parquet (columnar, comprimido)
- Transformación: dbt (DuckDB)
- Orquestación: Apache Airflow 2.10
- Contenedores: Docker Compose
- CI/CD: GitHub Actions
- Documentación: dbt docs + README

## Pipeline (diario 6:00 AM)
| Paso | Duración estimada |
|-----|-----|
| Extract | 2 min |
| Load to Bronze | 1 min |
| dbt Run | 5 min |
| dbt Test | 2 min |
| dbt Docs | 1 min |

```

```
| **Total** | **~11 min** |

## Estructura del Data Lake
- Bronze: datos crudos en Parquet (schema-on-read)
- Silver: datos limpios y validados
- Gold: datos modelados para BI (DuckDB)

## Servicios
| Servicio | Puerto | Credenciales |
|-----|-----|-----|
| Airflow | 8080 | admin / admin |
| MinIO API | 9000 | minioadmin / minioadmin123 |
| MinIO Console | 9001 | minioadmin / minioadmin123 |
```

Paso 4: Verificación final

```
# 1. Verificar servicios
docker compose ps
# Todos deben mostrar "Up"

# 2. Verificar DAGs en Airflow
curl http://localhost:8080/api/v1/dags -u admin:admin

# 3. Verificar datos en MinIO
curl http://localhost:9001/api/v1/buckets -u minioadmin:minioadmin123

# 4. Ejecutar pipeline manual
# Airflow UI → DAG → Trigger DAG

# 5. Verificar logs
# Airflow → DAG → ultima run → logs

# 6. Verificar dbt docs
# ls dbt/techstore_dbt/target/
```

Paso 5: Onboarding document

```
## Onboarding: Nuevo miembro del equipo TechStore Data

### Día 1
1. Clonar repo: `git clone https://github.com/techstore/data-platform`
2. Copiar `.env.example` → `.env` y configurar
3. `docker compose up -d`
4. Verificar Airflow: http://localhost:8080 (admin/admin)

### Día 2
1. Explorar DAGs existentes en Airflow
2. Leer modelos dbt en `dbt/techstore_dbt/models/`
3. Ejecutar `dbt docs serve` y explorar lineage
```

```

### Día 3
1. Hacer un cambio pequeño en un modelo dbt
2. Crear PR → esperar CI → mergear
3. Verificar deploy en Airflow

### Semana 1
1. Entender el pipeline completo
2. Hacer un backfill de un mes de datos
3. Agregar un test nuevo a un modelo existente

```

Entregables del Checkpoint 4

- ☐ Repositorio completo con estructura estándar
- ☐ CI/CD funcionando (GitHub Actions)
- ☐ Docker Compose con todos los servicios
- ☐ Pipeline orquestado en Airflow
- ☐ dbt models con tests y documentación
- ☐ Datos accesibles desde BI (Power BI / Looker)
- ☐ Ficha técnica del sistema documentada
- ☐ Onboarding document para nuevos miembros
- ☐ Runbook de operaciones

Checklist de calidad

Aspecto	Verificación
Datos correctos	dbt tests pasan (0 fallos)
Pipeline corre	Airflow muestra última run exitosa
CI/CD activo	GitHub Actions último PR verde
Docker	Todos los servicios "Up"
Documentación	dbt docs serve funcionando
Extract OK	Parquet en MinIO bronze
Transform OK	Gold accesible desde DuckDB
Reproducible	`git clone && docker compose up` funciona

Próximos pasos

1. **Conecta fuentes reales:** reemplaza SQLite por PostgreSQL, APIs, o streaming (Kafka)
2. **Escala a cloud:** migra MinIO a S3/GCS, DuckDB a Snowflake/BigQuery
3. **Agrega calidad:** Great Expectations para validación avanzada
4. **Real-time:** Kafka + Spark Streaming para datos en tiempo real
5. **Data Mesh:** distribuye la propiedad de datos entre equipos
6. **ML Pipeline:** agrega feature store y modelos ML orquestados

¡Felicidades! Has construido una Data Platform corporativa completa desde cero. Esto es lo que las empresas modernas usan para mover, transformar y orquestar datos a escala.

La colección **Data & Analytics Mastery** continúa:

Libro 5: Big Data, ML e IA: Especialización Avanzada

Conclusión

Del SQLite a la Data Platform

Has recorrido el camino completo de un ingeniero de datos moderno. Empezaste entendiendo qué es data engineering, construiste un data lake con MinIO y Parquet, transformaste datos con dbt, orquestaste pipelines con Airflow, empaquetaste todo en Docker, y finalmente implementaste CI/CD con GitHub Actions.

Lo que has logrado:

1. **Fundamentos de Data Engineering:** arquitectura medallion (bronze/silver/gold), rol del DE, stack moderno
2. **Python para DE:** logging, configuración, manejo de errores, decoradores, Path
3. **Formatos de datos:** Parquet, Avro, CSV — columnar vs row-oriented, particionamiento Hive-style
4. **Data Lakes:** MinIO, S3 API, object storage, schema-on-read
5. **dbt:** modelos SQL, sources, ref, materializaciones, testing, documentación, Jinja, macros, paquetes
6. **Apache Airflow:** DAGs, TaskFlow, XComs, sensores, BranchPythonOperator, backfills
7. **Docker:** contenedores, Dockerfile, Docker Compose, redes, volúmenes
8. **CI/CD:** GitHub Actions, tests automáticos, validación de DAGs, deploy

Habilidades adquiridas

Área	Skill
Lenguajes	Python (DE avanzado), SQL, Jinja, YAML
Data Lake	MinIO, Parquet, Avro, S3 API, boto3
Transformación	dbt (models, tests, docs, macros, packages)
Orquestación	Airflow (DAGs, TaskFlow, sensors, XComs)
Contenedores	Docker, Docker Compose, multi-stage builds
CI/CD	GitHub Actions, tests automatizados, deploy
Arquitectura	Medallion, data platform, pipelines, runbooks

¿Y ahora qué?

La colección **Data & Analytics Mastery** termina con:

Libro 5: Big Data, ML e IA: Especialización Avanzada

Mientras tanto, aquí tienes ideas para seguir practicando:

1. **Conecta fuentes reales** (APIs, PostgreSQL, Kafka) a tu data lake
2. **Migra a cloud**: MinIO S3, DuckDB Snowflake/BigQuery, Airflow MWAA/Cloud

Composer

3. **Implementa streaming**: Kafka + Spark Structured Streaming para datos en tiempo real
4. **Agrega calidad con Great Expectations**: validación avanzada de datos
5. **Construye un feature store**: para alimentar modelos de Machine Learning

Reflexión final

TechStore comenzó como un archivo SQLite en el Libro 1. Hoy tiene una Data Platform corporativa completa: data lake en MinIO, transformaciones con dbt, orquestación con Airflow, empaquetado en Docker, y despliegue automatizado con CI/CD.

La ingeniería de datos no es sexy. No produce dashboards bonitos ni modelos de ML impresionantes. Pero *sin ingeniería de datos, no hay dashboards, no hay ML, no hay analytics*. Eres la base sobre la que todo se construye.

Construye pipelines que no fallen. O que fallen bien. Y cuando fallen, que despierten a alguien.

Gracias por acompañarme en este viaje de 4 libros. Nos vemos en el Libro 5: *Big Data, ML e IA: Especialización Avanzada*.

— Alex Goyzueta Delgado

Apéndice A: Soluciones a los Ejercicios

Capítulo 1: Introducción al Data Engineering

Ejercicio 3 — ¿Por qué Parquet es mejor que CSV para un data lake?

1. **Tamaño:** Parquet comprime datos columnarmente (90% menos espacio)
2. **Velocidad:** solo lee las columnas necesarias
3. **Schema:** el schema está incrustado, no hay parsing de tipos
4. **Particionamiento:** soporte nativo Hive-style

Capítulo 2: Python para Data Engineering

Ejercicio 3 — Decorador @retry con backoff exponencial:

```
import time
from functools import wraps

def retry(max_attempts=5, base_delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts:
                        raise
                    delay = base_delay * (2 ** (attempt - 1))
                    print(f"Intento {attempt} falló. Reintentando en {delay}s...")
                    time.sleep(delay)
            return None
        return wrapper
    return decorator
```

Capítulo 3: Formatos de Datos

Ejercicio 1 — Comparación CSV vs Parquet:

```
import pandas as pd, sqlite3, os
```

```

conn = sqlite3.connect("data/techstore.db")
df = pd.read_sql_query("SELECT * FROM orders", conn)
conn.close()

df.to_csv("orders.csv", index=False)
df.to_parquet("orders.parquet", index=False)

csv_size = os.path.getsize("orders.csv")
parquet_size = os.path.getsize("orders.parquet")
print(f"CSV: {csv_size / 1024:.0f} KB")
print(f"Parquet: {parquet_size / 1024:.0f} KB")
print(f"Parquet es {csv_size / parquet_size:.1f}x más pequeño")

```

Capítulo 4: Data Lakes y MinIO

Ejercicio 6 — Descargar archivo y verificar integridad:

```

import boto3, hashlib

s3 = boto3.client("s3", endpoint_url="http://localhost:9000",
                  aws_access_key_id="minioadmin",
                  aws_secret_access_key="minioadmin123")

# Descargar
s3.download_file("techstore-bronze", "orders/orders_latest.parquet", "download.parquet")

# Verificar checksum (opcional)
md5 = hashlib.md5()
with open("download.parquet", "rb") as f:
    md5.update(f.read())
print(f"MD5: {md5.hexdigest()}")

```

Capítulo 5: Introducción a dbt

Ejercicio 2 — Staging model de customers:

```

-- models/staging/stg_customers.sql
{{ config(materialized='view') }}

SELECT
    customer_id,
    TRIM(name) AS customer_name,
    LOWER(TRIM(email)) AS email,
    TRIM(city) AS city,
    TRIM(country) AS country,
    CAST(registration_date AS DATE) AS registration_date
FROM {{ source('techstore', 'customers') }}

```

Capítulo 6: Modelado en dbt

Ejercicio 3 — Intermediate model int_product_sales:

```
-- models/intermediate/int_product_sales.sql
{{ config(materialized='view') }}

SELECT
    oi.product_id,
    p.product_name,
    p.category,
    COUNT(DISTINCT oi.order_id) AS pedidos,
    SUM(oi.quantity) AS unidades_vendidas,
    SUM(oi.line_total) AS ingresos
FROM {{ ref('stg_order_items') }} oi
JOIN {{ ref('stg_products') }} p ON oi.product_id = p.product_id
GROUP BY oi.product_id, p.product_name, p.category
```

Capítulo 7: Testing en dbt

Ejercicio 3 — Data test para totales negativos:

```
-- tests/assert_no_negative_totals.sql
SELECT order_id, total
FROM {{ ref('stg_orders') }}
WHERE total < 0
```

Capítulo 8: dbt Avanzado

Ejercicio 2 — Surrogate key con dbt_utils:

```
-- models/marts/fact_orders.sql
SELECT
    {{ dbt_utils.generate_surrogate_key(['order_id']) }} AS order_sk,
    order_id,
    customer_id,
    order_date,
    total,
    status
FROM {{ ref('stg_orders') }}
```

Capítulo 9: Introducción a Airflow

Ejercicio 2 — DAG con 3 tasks:

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG("simple_pipeline", schedule="@daily",
        start_date=datetime(2024, 1, 1), catchup=False) as dag:

    start = BashOperator(task_id="start", bash_command="echo 'Inicio'")
    process = BashOperator(task_id="process", bash_command="echo 'Procesando...'")
    end = BashOperator(task_id="end", bash_command="echo 'Fin'")
```

```
start >> process >> end
```

Capítulo 10: Airflow Avanzado

Ejercicio 2 — Sensor para Parquet en MinIO:

```
from airflow.sensors.base import BaseSensorOperator

class ParquetSensor(BaseSensorOperator):
    def __init__(self, bucket, prefix, **kwargs):
        super().__init__(**kwargs)
        self.bucket = bucket
        self.prefix = prefix

    def poke(self, context):
        import boto3
        s3 = boto3.client("s3", endpoint_url="http://minio:9000",
                           aws_access_key_id="minioadmin",
                           aws_secret_access_key="minioadmin123")
        resp = s3.list_objects_v2(Bucket=self.bucket, Prefix=self.prefix)
        files = [o for o in resp.get("Contents", []) if o["Key"].endswith(".parquet")]
        return len(files) > 0
```

Capítulo 11: Airflow + dbt

Ejercicio 4 — Notificación Slack en fallo:

```
from airflow.providers.slack.operators.slack_webhook import SlackWebhookOperator

def notify_slack_failure(context):
    SlackWebhookOperator(
        task_id="slack_alert",
        slack_webhook_conn_id="slack_default",
        message=f" Pipeline falló: {context['dag'].dag_id}.{context['task'].task_id}",
    ).execute(context=context)
```

Capítulo 12: Monitoreo

Ejercicio 3 — SLA de 30 minutos para dbt_run:

```
dbt_run = BashOperator(
    task_id="dbt_run",
    bash_command="dbt run",
    sla=timedelta(minutes=30),
)
```

Capítulo 13: Docker

Ejercicio 1 — Dockerfile para pipeline:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY pipelines/ ./pipelines/
COPY config/ ./config/
COPY run_pipeline.py .
CMD ["python", "run_pipeline.py"]
```

Capítulo 14: CI/CD

Ejercicio 1 — Workflow dbt compile en PR:

```
name: dbt Compile
on:
  pull_request:
    paths: ['dbt/**']
jobs:
  compile:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install dbt-core dbt-duckdb
      - run: dbt deps && dbt compile
      working-directory: dbt/techstore_dbt
```

Capítulo 15: Proyecto Final

Ejercicio 6 — Plan de disaster recovery:

1. **Backup diario** del warehouse DuckDB y configuración de Airflow
2. **docker compose down + docker compose up** para reinicio completo
3. **dbt snapshot** para datos que cambian (SCD2)
4. **Backfill** automático vía Airflow si se detecta data faltante
5. **Repositorio Git** como source of truth: todo es reproducible

Apéndice B: Cheatsheets de Data Engineering

Python para Data Engineering

```
# ===== Path =====
from pathlib import Path
BASE = Path(__file__).parent
Path("data/bronze").mkdir(parents=True, exist_ok=True)
for f in Path("data").rglob("*.parquet"):
    print(f.name)

# ===== Logging =====
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(name)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("techstore.pipeline")

# ===== Decoradores =====
from functools import wraps
def retry(max_tries=3, delay=2):
    def dec(f):
        @wraps(f)
        def wrapper(*a, **kw):
            for i in range(1, max_tries+1):
                try: return f(*a, **kw)
                except Exception as e:
                    if i == max_tries: raise
                    time.sleep(delay * i)
            return wrapper
        return dec
    return dec

# ===== Config =====
import yaml
with open("config/config.yaml") as f:
    config = yaml.safe_load(f)
```

Parquet

```
# ===== Escribir =====
import pandas as pd
df.to_parquet("data.parquet", index=False)

import pyarrow as pa
import pyarrow.parquet as pq
table = pa.Table.from_pandas(df)
pq.write_table(table, "data.parquet")

# ===== Particionado =====
df.to_parquet("data/orders/", partition_cols=["year", "month"])

# ===== Leer columnas específicas =====
df = pd.read_parquet("data.parquet", columns=["order_id", "total"])

# ===== Leer con DuckDB =====
import duckdb
df = duckdb.sql("SELECT * FROM 'data/*.parquet'").df()

# ===== Schema =====
schema = pa.schema([
    ("id", pa.int64()),
    ("name", pa.string()),
    ("price", pa.float64()),
])
```

MinIO (boto3)

```
import boto3
from botocore.config import Config

s3 = boto3.client("s3",
    endpoint_url="http://localhost:9000",
    aws_access_key_id="minioadmin",
    aws_secret_access_key="minioadmin123",
    config=Config(signature_version="s3v4"),
)

# Buckets
s3.create_bucket(Bucket="techstore-bronze")
s3.list_buckets()

# Upload / Download
s3.upload_file("local.parquet", "bucket", "path/remote.parquet")
s3.download_file("bucket", "path/remote.parquet", "local.parquet")

# List
resp = s3.list_objects_v2(Bucket="bucket", Prefix="orders/")
for obj in resp.get("Contents", []):
```

```

print(obj["Key"], obj["Size"])

# Delete
s3.delete_object(Bucket="bucket", Key="path/file.parquet")

```

dbt

```

# Comandos
dbt init project_name      # Crear proyecto
dbt run                    # Ejecutar modelos
dbt run --select model     # Modelo específico
dbt run --select staging+  # staging y dependientes
dbt test                   # Ejecutar tests
dbt test --select model    # Tests de un modelo
dbt deps                   # Instalar paquetes
dbt compile                # Compilar SQL sin ejecutar
dbt docs generate          # Generar docs
dbt docs serve             # Servir docs local
dbt snapshot               # Ejecutar snapshots
dbt debug                  # Verificar conexión

```

```

-- dbt SQL patterns
{{ config(materialized='table') }}
{{ config(materialized='incremental', unique_key='id') }}

FROM {{ source('schema', 'table') }}
JOIN {{ ref('other_model') }}

{% if is_incremental() %}
    WHERE date > (SELECT MAX(date) FROM {{ this }})
{% endif %}

```

```

# dbt_project.yml
models:
  project:
    staging:
      +materialized: view
      +schema: staging
  marts:
    +materialized: table

```

Airflow

```

# DAG básico
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG("dag_id", schedule="@daily",
        start_date=datetime(2024, 1, 1), catchup=False) as dag:

```

```

task1 = PythonOperator(task_id="t1", python_callable=my_func)
task2 = PythonOperator(task_id="t2", python_callable=my_func)
task1 >> task2

# TaskFlow
from airflow.decorators import dag, task
@dag(...)
def my_dag():
    @task
    def my_task():
        return "data"
    my_task()

# Operators comunes
BashOperator(task_id="t", bash_command="echo hi")
PythonOperator(task_id="t", python_callable=fn)
FileSensor(task_id="wait", filepath="/path/to/file")
BranchPythonOperator(task_id="branch", python_callable=fn)

```

Docker

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
USER nobody
CMD ["python", "main.py"]

```

```

# docker-compose.yml
services:
  app:
    build: .
    ports:
      - "8080:8080"
    volumes:
      - ./data:/app/data
    depends_on:
      - db
  db:
    image: postgres:15

```

GitHub Actions

```

name: CI
on:
  pull_request:
    branches: [main]
jobs:
  test:

```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: '3.11'
  - run: pip install -r requirements.txt
  - run: pytest tests/

```

Medallion Architecture

Bronze → Silver → Gold
 (crudo) (limpio) (analítico)

Bronze: datos tal cual de la fuente, Parquet
 Silver: datos validados, deduplicados, tipados
 Gold: hechos + dimensiones, listos para BI

Conceptos clave

Concepto	Definición
DAG	Directed Acyclic Graph – flujo de tareas sin loops
Medallion	Arquitectura de datos en 3 capas (bronze/silver/gold)
Schema-on-read	Schema se define al leer, no al escribir
Columnar	Datos almacenados por columna (Parquet) vs por fila (CSV)
Object Storage	Almacenamiento de objetos (S3, GCS, MinIO)
ELT	Extract → Load → Transform (vs ETL)
Idempotencia	Ejecutar N veces produce el mismo resultado
Backfill	Reprocesar datos históricos
Freshness	Qué tan actualizados están los datos
Lineage	Mapa de dependencias entre datos y transformaciones

Apéndice C: Instalación y Configuración

— Docker, Airflow, dbt

Instalación de Docker

Docker es requisito para MinIO, Airflow, y los pipelines.

Windows

1. Descarga Docker Desktop de <https://www.docker.com/products/docker-desktop>
2. Requisitos: WSL 2 habilitado, virtualización activada en BIOS
3. Instala y reinicia
4. Verifica:

```
docker --version
docker compose version
```

macOS

```
brew install --cask docker
# O descargar de https://www.docker.com/products/docker-desktop
```

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install docker.io docker-compose-v2
sudo usermod -aG docker $USER
# Cerrar sesión y volver a entrar
docker --version
```

Instalación de Python y dependencias

```
# Verificar versión
python --version # Debe ser 3.10+

# Crear entorno virtual (recomendado)
python -m venv venv
source venv/bin/activate # Linux/macOS
.\venv\Scripts\activate # Windows

# Instalar dependencias del proyecto
```

```
pip install -r requirements.txt
```

```
# requirements.txt (completo del libro)
pandas>=2.0
pyarrow>=15.0
boto3>=1.34
pyyaml>=6.0
dbt-core>=1.8
dbt-duckdb>=1.8
duckdb>=1.0
apache-airflow>=2.10
pytest>=8.0
ruff>=0.5
mypy>=1.10
slack-sdk>=3.27
```

Instalación de dbt

```
pip install dbt-core dbt-duckdb

# Verificar
dbt --version
# core: 1.8.x, duckdb: 1.8.x

# Iniciar proyecto
dbt init techstore_dbt
```

Configuración DuckDB

```
# ~/.dbt/profiles.yml
techstore_dbt:
  outputs:
    dev:
      type: duckdb
      path: /data/techstore_gold.duckdb
      schema: main
      threads: 4
    ci:
      type: duckdb
      path: /tmp/techstore_ci.duckdb
      schema: main
      threads: 2
    prod:
      type: duckdb
      path: /data/techstore_gold.duckdb
      schema: main
      threads: 8
  target: dev
```

Levantar Airflow con Docker

```
# Descargar docker-compose.yaml oficial
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/2.10.0/docker-compose.yaml'

# Crear directorios necesarios
mkdir -p dags logs plugins config

# Inicializar base de datos
docker compose up airflow-init

# Iniciar Airflow
docker compose up -d

# Acceder
# http://localhost:8080 (airflow/airflow)
```

Configuración personalizada

```
# docker-compose.override.yaml
services:
  airflow-webserver:
    ports:
      - "8080:8080"
    volumes:
      - ./dags:/opt/airflow/dags
      - ./data:/opt/airflow/data
      - ./dbt:/opt/airflow/dbt
  airflow-scheduler:
    volumes:
      - ./dags:/opt/airflow/dags
      - ./data:/opt/airflow/data
      - ./dbt:/opt/airflow/dbt
```

Troubleshooting

Docker: "Cannot connect to the Docker daemon"

- Windows: asegúrate que Docker Desktop esté corriendo
- Linux: `sudo systemctl start docker`
- Verificar: `docker ps`

MinIO: "Access Denied"

- Verificar credenciales (MINIO_ROOT_USER / MINIO_ROOT_PASSWORD)
- Verificar signature_version: "s4v4"
- MinIO requiere firma v4

dbt: "Could not find profile"

- Verificar que `profiles.yml` existe en `~/dbt/` o `DBT\_PROFILES\_DIR`
- Ejecutar `dbt debug` para diagnosticar

Airflow: "DAG import errors"

- Verificar sintaxis del DAG: ``python dags/mi_dag.py``
- Verificar dependencias instaladas en el contenedor
- Revisar logs del scheduler

Airflow: ModuleNotFoundError

- Los operadores adicionales requieren ``pip install apache-airflow-providers-*``
- Ej: ``apache-airflow-providers-slack``, ``apache-airflow-providers-docker``

GitHub Actions: "dbt not found"

- Agregar ``pip install dbt-core dbt-duckdb`` en el workflow
- O usar ``dbt-labs/dbt-action@v1``

Recursos adicionales

- **Docker:** <https://docs.docker.com>
- **MinIO:** <https://min.io/docs/minio/container/index.html>
- **dbt:** <https://docs.getdbt.com>
- **dbt Package Hub:** <https://hub.getdbt.com>
- **Airflow:** <https://airflow.apache.org/docs>
- **Airflow Providers:** <https://registry.astronomer.io>
- **DuckDB:** <https://duckdb.org/docs>
- **Parquet:** <https://parquet.apache.org/docs>
- **GitHub Actions:** <https://docs.github.com/actions>
- **Great Expectations:** <https://greatexpectations.io> (para calidad de datos)
- **Data Engineering Cookbook:** <https://github.com/andkret/Cookbook>

Sobre el Autor

Alex Goyzueta Delgado es analista de datos, instructor y emprendedor tecnológico con más de una década de experiencia transformando datos en decisiones de negocio.

Su trayectoria abarca desde bases de datos relacionales y hojas de cálculo hasta plataformas cloud empresariales, data engineering moderno y Business Intelligence, trabajando con startups y corporaciones en Latinoamérica para implementar soluciones de datos que generan impacto real.

Fundador de **Data & Analytics Academy**, donde ha formado a más de 5000 estudiantes en análisis de datos, SQL, Python, cloud computing, BI y data engineering a través de cursos prácticos y project-based learning.

Cree firmemente en que la mejor forma de aprender datos es construyendo proyectos reales desde el día uno. Su metodología combina fundamentos técnicos sólidos con casos de negocio prácticos, preparando a sus estudiantes no solo para usar herramientas, sino para pensar como profesionales de datos.

Este libro es el cuarto de la colección **Data & Analytics Mastery**, una serie diseñada para llevar al lector desde cero hasta analytics professional.

Conéctate con el autor:

- LinkedIn: <https://www.linkedin.com/in/alexgoyzueta>
- GitHub: <https://github.com/alexgoyzueta>

Otros libros de la colección:

1. *Los Fundamentos del Analista de Datos* — Domina SQL, Excel y Python desde cero
2. *Python para Datos y tu Primer Data Warehouse Cloud* — De Pandas a BigQuery
3. *Business Intelligence y Modelado de Datos* — Diseña dashboards profesionales
4. *Ingeniería de Datos Moderna* — Este libro
5. *Big Data, ML e IA: Especialización Avanzada* — Próximamente